Laporan Tugas Besar

BNPB Chisler

IF4031 - Arsitektur Aplikasi Terdistribusi 2026/2027



Disusun oleh :

Kelompok Chisli1

Varel Tiara	13523008
Bryan Ho	13523029
Kenneth Poenadi	13523040

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESHA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi **2**

Tabel Kontribusi **4**

BAB 1
Pendahuluan **5**

1.1 Deskripsi Utama 5

1.2 Lingkup Milestone 1 5

1.3 Ringkasan Problem 6

BAB 2
Arsitektur Sistem **7**

2.1 Diagram Arsitektur 7

2.2 System Flow 8

2.3 Tanggung Jawab per Service 10

2.4 Observabilitas Sistem 11

BAB 3
Daftar Teknologi **12**

BAB 4
Asumsi **13**

4.1 Asumsi 13

BAB 5
Problem 1: Arsitektur dan Interoperabilitas Data **15**

5.1 Rancangan 15

5.2 Alasan dan Alternatif 17

5.3 Batasan yang disadari 17

5.4 Bukti Penyelesaian 18

5.5 Jawaban Pertanyaan 24

BAB 6
Problem 2: Concurrency, Availability, and Graceful Degradation **25**

6.1 Rancangan 25

6.2 Alasan dan Alternatif 27

6.3 Batasan yang disadari 28

6.4 Bukti Penyelesaian 29

6.5 Jawaban Pertanyaan 34

BAB 7
Problem 3: Service to Service Authentication and Trust **36**

7.1 Rancangan 36

7.2 Alasan dan Alternatif 39

7.3 Batasan yang disadari 40

7.4 Bukti Penyelesaian 40

7.5 Jawaban Pertanyaan 49
BAB 8
Problem 4: Arsitektur Microservice dan Flexible Storage 51
8.1 Rancangan 51
8.2 Alasan dan Alternatif 52
8.3 Batasan yang disadari 53
8.4 Bukti Penyelesaian 53
8.5 Jawaban Pertanyaan 60
BAB 9
Problem 5: Fan-out Event Distribution/Pub-sub 61
9.1 Rancangan 61
9.2 Alasan dan Alternatif 62
9.3 Batasan yang disadari 63
9.4 Bukti Penyelesaian 63
9.5 Jawaban Pertanyaan 69
BAB 10
Petunjuk Menjalankan 71
10.1 Langkah Menjalankan 71
10.2 Daftar Port 71
10.3 Memicu Scenario 72
LAMPIRAN 74
A. Deklarasi Penggunaan LLM 74
B. Tautan Repositori dan Tag 74
C. All Output 74

Tabel Kontribusi

NIM	Nama	Pembagian Tugas
13523008	Varel Tiara	Mock BMKG dan PVMBG, load testing dengan k6, Problem 4 (service terpisah dalam container, MongoDB sebagai Canonical Store, Docker Compose), daftar teknologi, penyatuan laporan
13523029	Bryan Ho	Problem 3 (auth-service, scope client, refresh token, penegakan hak akses di client-facing-api), pendahuluan, asumsi, petunjuk menjalankan, penulisan laporan
13523040	Kenneth Poenadi	Problem 1, 2, dan 5 (aggregator, polling, pemetaan, degradasi saat sumber mati, RabbitMQ dan consumer), arsitektur sistem, penulisan laporan

Tabel 1. Tabel Kontribusi Mahasiswa

BAB 1
Pendahuluan

1.1 Deskripsi Utama

Sistem yang dibangun merupakan platform koordinasi informasi kebencanaan yang menjembatani dua instansi sumber data, yaitu BMKG (Badan Meteorologi, Klimatologi, dan Geofisika) dan PVMBG (Pusat Vulkanologi dan Mitigasi Bencana Geologi), dengan BNPB (Badan Nasional Penanggulangan Bencana) sebagai node agregasi dan distribusi.

BMKG dan PVMBG masing-masing mempublikasikan data kebencanaan sesuai domainnya. BMKG untuk kejadian seismik (gempa bumi) dan peringatan tsunami, PVMBG untuk laporan aktivitas vulkanik. Kedua instansi ini diimplementasikan sebagai *mock service* yang mengikuti kontrak API tetap (skema data, endpoint, autentikasi, dan karakteristik latensi) agar seluruh kelompok diuji terhadap target yang setara.

BNPB merupakan komponen utama yang dikembangkan yang bertugas melakukan *polling* berkala terhadap kedua sumber, menormalisasi data mentah dari masing-masing instansi ke dalam satu format kanonik (HazardEvent), menyimpannya secara terpusat, serta menyediakan akses data tersebut kepada *client* eksternal dengan kontrol akses berbasis identitas dan *scope*. Selain menyajikan data secara *on-demand* melalui API, BNPB juga menyiarkan setiap kejadian baru kepada pihak-pihak yang berlangganan secara *real-time* melalui mekanisme publish-subscribe.

Seluruh komunikasi antar *service* dilakukan murni melalui jaringan (REST/gRPC/*message broker*), tanpa pemanggilan fungsi langsung maupun *library business-logic* yang dibagikan antar *service*, dan setiap komponen berjalan sebagai *container* independen yang dapat dihentikan, dibangun ulang, dan dijalankan kembali tanpa memengaruhi komponen lain.

1.2 Lingkup Milestone 1

Lingkup implementasi pada Milestone 1 dibatasi sebagai berikut:

- **Dua instansi sumber data:** BMKG dan PVMBG, masing-masing dengan skema data, mekanisme autentikasi, dan karakteristik latensi yang berbeda dan saling tidak dapat dipertukarkan.

- **Dua jenis bahaya:** kejadian seismik/tsunami (dari BMKG) dan laporan vulkanik (dari PVMBG), yang keduanya dipetakan ke satu format kanonik HazardEvent oleh Aggregator.
- **BNPB sebagai node koordinasi tunggal:** seluruh data yang disajikan ke *client* eksternal maupun yang disiarkan ke *consumer* internal berasal dari satu sumber kebenaran (*canonical store*) yang dimiliki dan dikelola oleh Aggregator, tidak ada komponen lain yang mengakses *database* ini secara langsung.
- **Seluruh *client* bersifat *read-only*:** tiga jenis *client* eksternal (Media/Pers, Tim Lapangan, dan BNPB Pusat/Internal Ops) hanya dapat membaca data sesuai *scope* aksesnya masing-masing, tidak ada operasi tulis dari sisi *client*.
- **Jalur pelaporan dari lapangan tidak termasuk dalam lingkup:** skenario di mana Tim Lapangan mengirimkan data atau laporan baru ke sistem (*write path*) belum diimplementasikan.

1.3 Ringkasan Problem

Kelima problem yang diselesaikan pada Milestone 1 dapat diringkas sebagai berikut:

1. **Problem 1: Arsitektur & Interoperabilitas Data**
   Sistem mampu menangani penambahan field baru pada skema sumber data (PVMBG) secara dinamis saat sistem berjalan, tanpa memerlukan *restart*, *redeploy*, maupun perubahan kode pada Aggregator.
2. **Problem 2: *Concurrency*, *Availability*, & *Graceful Degradation***
   Aggregator melakukan *polling* paralel terhadap kedua sumber dengan batas konkurensi dan timeout yang terkendali, sehingga lambat atau matinya satu sumber tidak menghambat permintaan yang bergantung pada sumber lain maupun menjatuhkan seluruh sistem
3. **Problem 3: *Service-to-Service Authentication & Trust***
   Setiap instansi dan setiap jenis *client* memiliki identitas serta kredensial yang terpisah dan tidak dapat saling dipertukarkan, dengan kontrol akses berbasis *scope* yang ditegakkan di sisi *server*, bukan hanya disembunyikan di tampilan.
4. **Problem 4: Arsitektur Microservice & *Flexible* Storage**
   Setiap komponen BNPB berjalan sebagai layanan independen dalam kontainer terpisah, dengan penyimpanan kanonik yang mampu menampung perubahan struktur data (*field* baru) tanpa migrasi skema.
5. **Problem 5: *Fan-out Event Distribution/Pub-sub***
   Setiap kejadian bahaya baru disiarkan melalui *message broker* kepada *multiple consumer* independen secara bersamaan, sehingga penambahan *consumer* baru tidak memerlukan perubahan kode pada sisi *producer* (Aggregator).

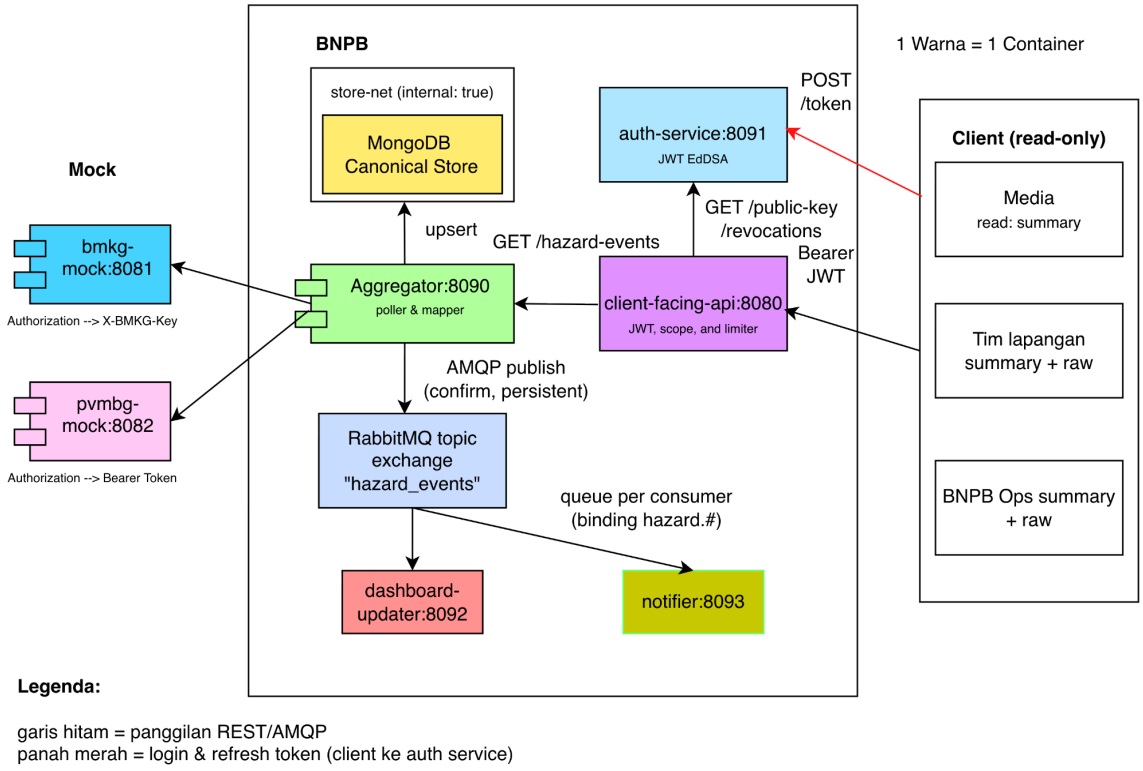
BAB 2

Arsitektur Sistem

2.1 Diagram Arsitektur

Sistem terdiri dari sembilan *container* yang di orkestrasi dengan Docker Compose, dua mock instansi sumber (bmkg-mock dan pvmbg-mock), tujuh komponen BNPB (aggregator, MongoDB, RabbitMQ, auth-service, client-facing-api, dashboard-updater, dan notifier).

Gambar 1 menunjukkan seluruh service, protokol komunikasi, kepemilikan penyimpanan, alur event, dan batas container.



Gambar 1. Diagram Arsitektur Sistem BNPB

Penjelasan naratif diagram:

- Aggregator *pulling* data (*polling*), bukan menerima *push*. Aggregator memanggil BMKG (header X-BMKG-Key) dan PVMBG (header Authorization: Bearer) secara berkala lewat REST. Kedua mock bersifat pasif dan tidak pernah mengirim apapun ke BNPB.
- *Client* tidak pernah menyentuh *mock*. *Client* hanya berbicara dengan client-facing-api, yang memverifikasi *token*, menegakkan *scope*, dan membaca data lewat API aggregator. Karena itu BNPB tetap dapat menjawab walau PVMBG mati.

- Canonical Store (MongoDB) hanya dimiliki aggregator. MongoDB berada di jaringan store-net yang bersifat *internal* (internal: true), tidak mempunyai port ke host, dan memakai autentikasi. Service lain memperoleh data lewat GET /hazard-events milik aggregator.

- Aggregator tidak mengetahui siapa *consumer*. Setiap HazardEvent hanya dipublikasikan ke *topic exchange* hazard_events di RabbitMQ. Tiap *consumer* (dashboard-updater, notifier) memiliki *queue* sendiri yang terikat ke *exchange* tersebut.

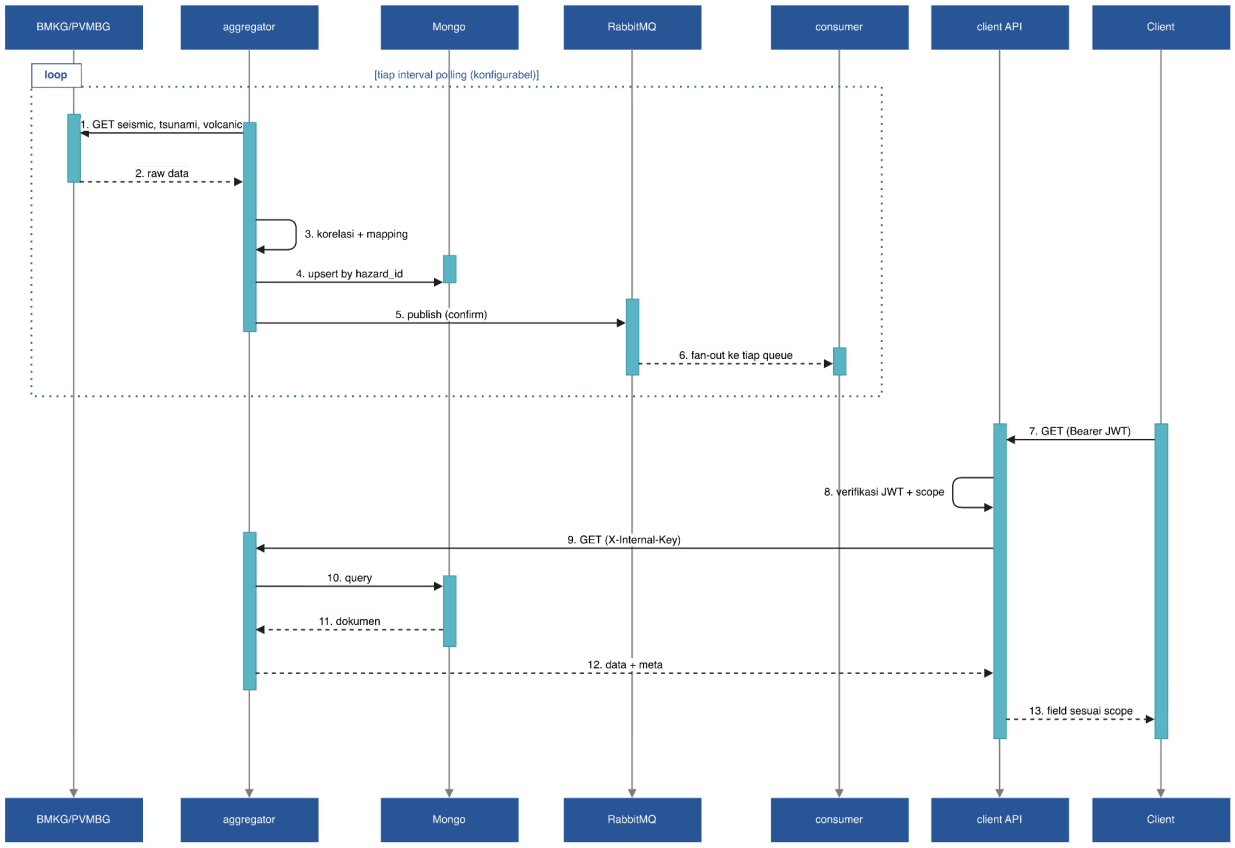
- auth-service adalah *domain trust* tersendiri. Ia menerbitkan JWT (EdDSA) untuk tiga jenis *client*. client-facing-api hanya mengambil *public key* dan daftar *revocation* dari auth-service, sehingga dapat memverifikasi *token* secara lokal tanpa dapat memalsukannya.

Jalur	Protokol	Autentikasi
aggregator ke bmkg-mock	REST (HTTP), polling	X-BMKG-Key
aggregator ke pvmbg-mock	REST (HTTP), polling	Authorization: Bearer <PVMBG_TOKEN>
aggregator ke MongoDB	Protokol Mongo (jaringan store-net)	Username dan password Mongo
aggregator ke RabbitMQ	AMQP, *publisher confirm*	Kredensial RabbitMQ dari .env
RabbitMQ ke consumer	AMQP, ack manual	Kredensial RabbitMQ dari .env
client-facing-api ke aggregator	REST	X-Internal-Key (AGGREGATOR_API_KEY)
client-facing-api ke auth-service	REST (/public-key, /revocations)	X-Internal-Key (AUTH_INTERNAL_KEY) untuk /revocations
client ke auth-service	REST (POST /token)	client_id dan client_secret, atau *refresh token*
client ke client-facing-api	REST	Authorization: Bearer <JWT>

Tabel 2. Protokol dan autentikasi pada setiap jalur komunikasi

2.2 System Flow

Alur sistem terbagi dua dan berjalan tidak saling bergantung: alur ingest dan alur serve (data disajikan saat client bertanya). Gambar 2 merangkum keduanya.



Gambar 2. System flow: ingest (polling, mapping, simpan, publish) dan serve (permintaan client)

Alur *ingest* (satu siklus *polling* BMKG):

1. Poller membuat *correlation* ID baru dan mencatat waktu *now* sebelum *request*, supaya tidak ada celah waktu antar siklus.

2. GET /seismic-events?since=. Cursor kosong pada *polling* pertama sehingga seluruh *seed* (20 *record* historis) terambil.

3. GET /tsunami-warnings?since=. Jika langkah ini gagal, seluruh siklus dilewati dan cursor tidak maju, karena memetakan *event* tsunami tanpa warning menghasilkan *severity* yang salah.

4. Aggregator membangun peta related_event_id ke warning untuk mengorelasikan SeismicEvent dengan TsunamiWarning. Bila ada *warning* yang *event*nya sudah di *ingest* sebelumnya, *event* itu diambil ulang, dipetakan ulang dengan *severity* yang benar, lalu dipublikasikan ulang.

5. Setiap *event* dipetakan ke HazardEvent, lalu disimpan ke MongoDB (upsert) dan dipublikasikan ke RabbitMQ dengan menunggu konfirmasi broker. *Record* yang tidak valid dilewati dan dicatat di *log*.

6. Bila ada kegagalan simpan atau *publish*, cursor tidak dimajukan sehingga *batch* dicoba lagi pada siklus berikutnya. Bila sukses, cursor dimajukan ke now. Siklus PVMBG serupa tetapi tanpa

korelasi *warning*. Bila PVMBG gagal (outage 503, timeout, atau galat), poller menandai *status* PVMBG DEGRADED, mencatat stale_since, dan kembali tanpa memajukan cursor. Setelah PVMBG pulih, status kembali HEALTHY.

Alur *serve* (permintaan client):

1. Client mengambil *token* dari auth-service, lalu memanggil GET /hazard-events pada client-facing-api dengan Authorization: Bearer.
2. client-facing-api memverifikasi tanda tangan dan klaim token, memeriksa scope, dan mengambil *slot* pada *limiter* konkurensi.
3. client-facing-api meneruskan permintaan ke aggregator (dengan X-Internal-Key dan X-Correlation-ID), yang membaca dari MongoDB tanpa memanggil BMKG atau PVMBG.
4. client-facing-api memproyeksikan *field* sesuai *scope* (*media* hanya mendapat *field summary*) sebelum menjawab

2.3 Tanggung Jawab per Service

Tabel berikut merangkum tugas tiap *service* beserta alasan *service* tersebut dipisah.

Service	Port	Tanggung jawab	Alasan dipisah
bmkg-mock	8081	Simulasi BMKG: /seismic-events, /tsunami-warnings, /health. Delay tetap per *instance* (BMKG_DELAY_MS, default 100, dikunci 50–150 ms). Validasi X-BMKG-Key. Membuat *event* baru tiap 10 detik.	Entitas luar, kontrak API tidak boleh diubah.
pvmbg-mock	8082	Simulasi PVMBG: /volcanic-reports, /admin/schema-version, /admin/outage, /health. Delay acak antara PVMBG_DELAY_MIN_MS dan PVMBG_DELAY_MAX_MS (default 500–3000, kode menolak nilai di luar 500 ≤ min ≤ max ≤ 3000, sehingga delay tetap 3 dtk berarti keduanya 3000). Validasi Authorization: Bearer.	Entitas luar, sengaja lambat dan dapat dimatikan untuk menguji P2.
aggregator	8090 (hanya 127.0.0.1 di host)	Polling, korelasi *warning*, mapping ke HazardEvent, simpan ke MongoDB, publish ke RabbitMQ, serta API baca internal (/hazard-events, /pvmbg-status).	Satu-satunya pemilik data kanonik, ingest, mapping, dan penyimpanan berubah bersamaan.
MongoDB	internal	Canonical Store.	Penyimpanan fleksibel untuk atribut dinamis (P4).

RabbitMQ	5672 (UI 15672, localhost)	*Message broker* dengan *topic exchange* hazard_events.	Memisahkan *producer* dari *consumer* (P5).
auth-service	8091	Menerbitkan JWT (*access* dan *refresh*), rotasi *refresh token*, dan daftar *revocation*.	Domain *trust* sendiri mengganti aturan auth tidak memaksa *rebuild service* lain (P3).
client-facing-api	8080	Pintu masuk *client*: verifikasi token, penegakan *scope*, proyeksi *field*, *limiter* konkurensi.	Permukaan serang menghadap luar dan profil beban berbeda (P2, P3).
dashboard-updater	8092 (health, hanya di jaringan Compose)	*Consumer* #1: menyimpan tampilan terbaru per hazard_id, dapat dilihat di /state.	*Consumer* independen (P5).
notifier	8093 (health, hanya di jaringan Compose)	*Consumer* #2: mensimulasikan pengiriman peringatan bila *severity* mencapai ambang (default WASPADA).	*Consumer* independen (P5).

Tabel 3. Tanggung jawab per service

2.4 Observabilitas Sistem

- **Health check.** Setiap *service* menyediakan GET /health. Docker Compose memakainya sebagai healthcheck dan depends_on: condition: service_healthy untuk mengatur urutan nyala. Aggregator memeriksa *ping* MongoDB dan status RabbitMQ (503 hanya bila MongoDB mati). *Consumer* melaporkan 503 bila terputus dari RabbitMQ.

- **Log terstruktur.** Seluruh *service* menulis satu baris JSON per *log* (slog JSON *handler*, atau format JSON manual pada *mock*) sehingga dapat difilter dengan jq atau grep.

- C**orrelation ID antar *service*.** Poller membuat ID baru pada tiap siklus, mengirimnya ke *mock* lewat header X-Correlation-ID, memasangnya ke properti CorrelationId pesan RabbitMQ, dan *consumer* mencatatnya dari msg.CorrelationId. Untuk *request* client, client-facing-api membaca X-Correlation-ID dari client (atau membuatnya) dan meneruskannya ke aggregator. Dengan demikian satu grep <id> pada seluruh *container* menunjukkan perjalanan satu *event* dari *mock* hingga *consumer*.

- ***Latency*.** Setiap panggilan keluar dicatat dengan target, operasi, duration_ms, status keberhasilan, dan correlation ID: aggregator ke BMKG, PVMBG, MongoDB (simpan, baca, ping, koneksi), dan RabbitMQ (publish, koneksi), client-facing-api ke aggregator dan auth-service (kunci publik, sinkron daftar token yang dicabut), serta koneksi kedua consumer ke RabbitMQ.

- ***Configuration*.** Seluruh kredensial, port, dan parameter tuning dibaca dari" → "Kredensial, port, dan parameter utama dibaca dari". Di akhir butir tambah: "Konstanta tuning internal (batas login 10 per menit, ukuran dedup 10.000, timeout dan jumlah retry koneksi, batas limit, ukuran body /token) ada di kode.

BAB 3

Daftar Teknologi

Teknologi yang digunakan pada sistem dipilih berdasarkan kebutuhan arsitektur pada Milestone 1, khususnya independensi antar-service, fleksibilitas terhadap perubahan skema data, distribusi event secara asynchronous, pemisahan autentikasi, serta kebutuhan pengujian concurrency dan availability.

Komponen	Teknologi	Alasan
Bahasa Pemrograman	Go (Golang)	Seluruh service dikembangkan menggunakan Go agar setiap komponen dapat dibangun dan dijalankan secara independen. Dukungan concurrency yang dimiliki Go pada goroutine sesuai dengan kebutuhan sistem untuk melakukan polling sumber data, menangani request HTTP, serta menjalankan proses asynchronous dengan overhead yang relatif kecil.
Canonical Store	MongoDB	Data HazardEvent memiliki field kanonik yang stabil serta attributes yang dapat menampung field tambahan dari sumber. Model dokumen BSON pada MongoDB memungkinkan field baru seperti confidence_level disimpan tanpa membutuhkan perubahan skema atau migrasi database.
Message Broker	RabbitMQ	RabbitMQ digunakan untuk mendistribusikan HazardEvent dari Aggregator ke beberapa consumer secara asynchronous. Penggunaan topic exchange dan queue terpisah per consumer memungkinkan penambahan consumer tanpa mengubah producer, serta mendukung mekanisme acknowledgement dan durable queue untuk semantik pengiriman at-least-once.
Authentication Token	JWT dengan EdDSA	Auth-service menandatangani access token menggunakan private key, sedangkan client-facing-api melakukan verifikasi menggunakan public key. Mekanisme ini memungkinkan token diverifikasi secara lokal tanpa melakukan request ke auth-service untuk setiap akses, sehingga mengurangi coupling antar-service.
Orkestrasi	Docker Compose	Setiap komponen dijalankan sebagai container terpisah dan diorkestrasi menggunakan Docker Compose. Docker Compose digunakan untuk mengatur dependency, health check, konfigurasi environment, jaringan antar-service, serta memungkinkan satu service dibangun atau dijalankan ulang secara independen tanpa harus me-restart service lainnya.
Load Test	k6	k6 digunakan untuk menguji perilaku sistem pada kondisi request concurrent dan beban yang berjalan dalam periode tertentu. Pengujian menghasilkan metrik seperti throughput, p50, p95, p99, error rate, serta jumlah response 429, sehingga dapat digunakan untuk mengevaluasi concurrency, latency, dan graceful degradation sistem.

Tabel 4. Daftar teknologi

BAB 4

Asumsi

4.1 Asumsi

Berikut asumsi yang diambil dalam perancangan dan implementasi:

1. Kedua mock bersifat pasif. Mock hanya merespons permintaan dan tidak pernah melakukan *push* ke Aggregator. Aggregator selalu menjadi pihak yang memulai komunikasi lewat polling.

2. Setiap mock menyediakan minimal 20 record historis saat pertama dinyalakan, sehingga sistem tidak dimulai dari keadaan kosong.

3. Tabel referensi koordinat gunung api (volcano_id ke nama dan koordinat) bersifat statis milik BNPB. Tabel ini dikodekan langsung di Aggregator, bukan diambil dari PVMBG atau sumber eksternal lain.

4. hazard_id diturunkan deterministik dari source_ref_id dengan format hz-<source_ref_id>, bukan UUID acak. Dengan demikian, *polling* ulang terhadap *record* yang sama selalu menghasilkan hazard_id yang identik, dan penyimpanan bersifat idempoten (*upsert*).

5. Jam seluruh *container* dianggap sinkron, yaitu memakai jam host Docker yang sama, tanpa mekanisme NTP terpisah antar *container*. Asumsi ini relevan untuk perbandingan occurred_at/ingested_at dan korelasi waktu pada log.

6. Hanya BMKG dan PVMBG yang menjadi sumber data. Sistem tidak dirancang untuk menerima instansi sumber ketiga tanpa perubahan kode pada *client* dan *mapper*.

7. Kebijakan *field* tak dikenal adalah diteruskan ke *attributes*, bukan diabaikan. Kebijakan ini berlaku konsisten untuk kedua sumber, termasuk confidence_level saat pertama muncul.

8. Evolusi skema yang didukung terbatas pada penambahan *field* baru. Perubahan tipe data atau penghapusan *field* wajib pada *field* yang sudah dikenal dianggap kegagalan pemetaan, sehingga *event* tersebut dilewati dan tidak dipaksakan masuk.

9. Tiga jenis *client* (Media, Tim Lapangan, Internal Ops) bersifat tetap dan statis di auth-service. Masing-masing memiliki satu identitas dan satu *scope*, dan tidak ada pendaftaran *client* baru secara dinamis.

10. Status sesi dan *refresh* token disimpan di memori proses auth-service, bukan di *database* persisten. Akibatnya, restart layanan ini menghapus seluruh sesi aktif.

11. Daftar token yang dicabut disinkronkan ke client-facing-api secara berkala lewat *polling*, bukan secara instan. Ada jendela singkat ketika token yang baru dicabut masih bisa diterima sebelum sinkronisasi berikutnya.

12. Semantik pengiriman event adalah *at least once*, bukan *exactly once*. Oleh karena itu, setiap *consumer* diasumsikan harus idempoten dengan melakukan deduplikasi berdasarkan hazard_id dan severity.

13. Seluruh sistem dijalankan dalam satu host Docker Compose. Tidak ada asumsi mengenai perilaku pada orkestrasi *multi host*, dan Mongo, RabbitMQ, serta Aggregator masing masing berjalan sebagai instans tunggal.

BAB 5

Problem 1: Arsitektur dan Interoperabilitas Data

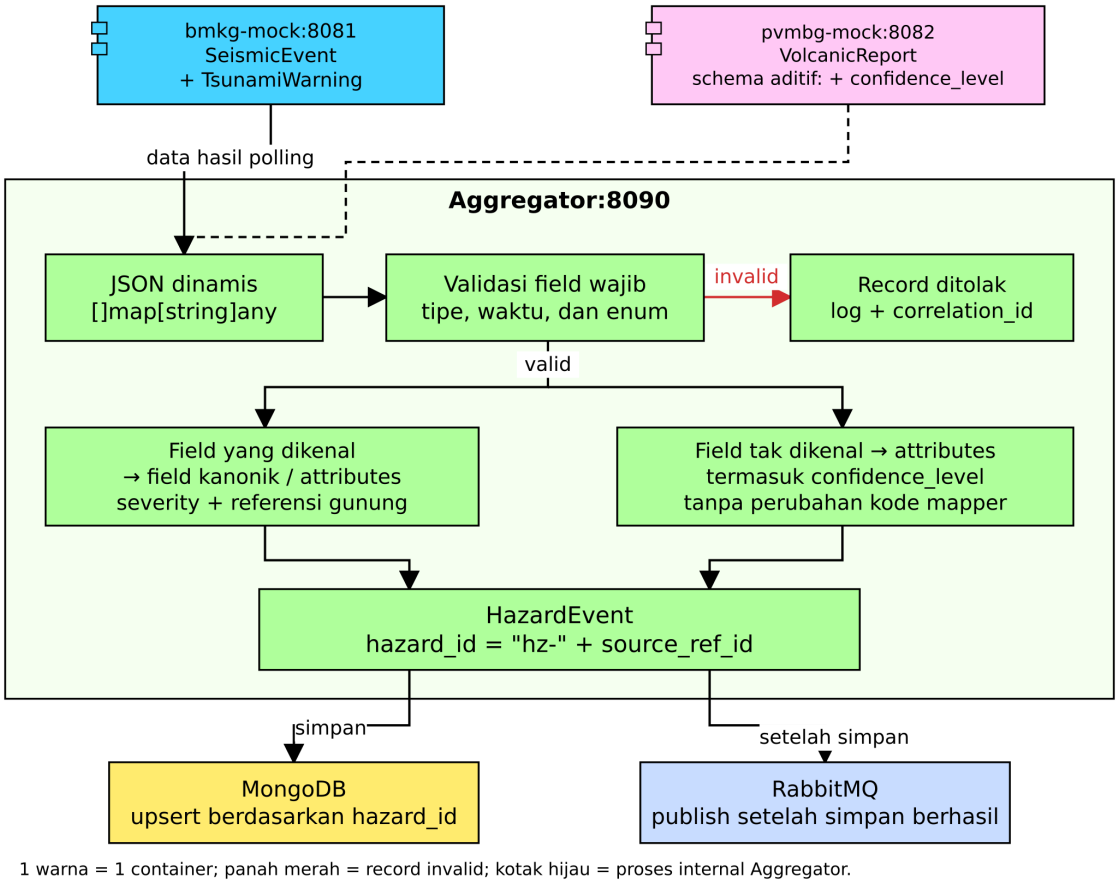
5.1 Rancangan

Masalah

Gateway lama bergantung pada bentuk data sumber yang tetap. Ketika PVMBG menambahkan *field* baru seperti confidence_level, informasi tersebut dapat hilang atau proses *mapping* gagal. Sistem perlu menerima perubahan skema aditif saat berjalan, mempertahankan *field* tambahan, dan tetap menghasilkan HazardEvent yang valid tanpa *restart*, *redeploy*, atau perubahan kode mapper.

Mekanisme mapping dinamis

Aggregator menerima JSON sumber sebagai map[string]any, memvalidasi *field* wajib, lalu memetakan *record* ke HazardEvent. Field tambahan dipertahankan di attributes sehingga perubahan skema aditif dapat diterima tanpa perubahan kode mapper.



Gambar 3. Alur mapping dinamis pada Aggregator

Kebijakan yang dideklarasikan

- Field tak dikenal diteruskan ke attributes dan tidak diabaikan.
- Field wajib divalidasi sebelum mapping. Jika field hilang atau tidak dapat dipetakan, record ditolak dan galat dicatat bersama correlation_id, service tidak crash dan record berikutnya dalam batch tetap diproses.

Aturan mapping

Aturan mengikuti spesifikasi. Untuk SeismicEvent, severity mengikuti threat_level dari TsunamiWarning terkait bila tersedia, jika tidak, severity ditentukan dari magnitudo: kurang dari 5,0 menjadi NORMAL, 5,0 sampai kurang dari 6,5 menjadi WASPADA, dan minimal 6,5 menjadi SIAGA. Untuk VolcanicReport, alert_level dipetakan langsung ke severity. hazard_id dibentuk secara deterministik sebagai "hz-" + source_ref_id sehingga pemrosesan ulang memperbarui dokumen yang sama melalui upsert.

Kanonik	Dari SeismicEvent (BMKG)	Dari VolcanicReport (PVMBG)
hazard_id	"hz-" + event_id	"hz-" + report_id
source / hazard_type	BMKG / SEISMIC	PVMBG / VOLCANIC
source_ref_id	event_id	report_id
area_name	region_name	Nama gunung dari tabel referensi volcano_id
latitude, longitude	epicenter_lat, epicenter_lon	Koordinat dari tabel referensi statis BNPB
occurred_at	occurred_at	reported_at
ingested_at	Waktu BNPB menerima data	Waktu BNPB menerima data
severity	TsunamiWarning.threat_level, jika tidak ada, dari magnitudo	alert_level dipetakan langsung
attributes	magnitude, depth_km, potential_tsunami, field warning berprefiks tsunami_warning_, dan field tak dikenal	eruption_count_24h, ash_column_height_m, dan field tak dikenal termasuk confidence_level

Tabel 5. Aturan mapping ke HazardEvent

Tabel referensi gunung milik BNPB memuat G-001 Gunung Merapi, G-002 Gunung Kelud, G-003 Gunung Semeru, dan G-004 Gunung Sinabung. Jika volcano_id tidak dikenal, area_name menjadi "Unknown Volcano (<id>)", koordinat diisi 0, dan record tetap diterima.

Perubahan skema pada mock PVMBG

POST /admin/schema-version dengan body {"enabled": true} memanggil Store.SetSchemaEnabled pada pvmbg-mock/internal/pvmbg/store.go. Setelah diaktifkan, setiap report baru memiliki confidence_level pada rentang 0,70–1,00. Mock langsung membuat satu report baru agar perubahan segera terlihat, sedangkan report lama tidak berubah. Perubahan ini tidak me-restart mock maupun Aggregator.

Penyimpanan dan publikasi

HazardEvent disimpan ke MongoDB menggunakan upsert berdasarkan hazard_id, kemudian dipublikasikan oleh Aggregator ke RabbitMQ setelah penyimpanan berhasil. Field confidence_level masuk ke attributes karena tidak termasuk daftar field yang dikenal. Implementasi mapping terdapat pada services/aggregator/internal/mapper/seismic.go, volcanic.go, dan helpers.go.

5.2 Alasan dan Alternatif

Alasan memilih JSON dinamis

JSON dinamis dipilih karena bentuk respons BMKG dan PVMBG berbeda, sementara *field* tambahan dari sumber perlu tetap tersedia bagi BNPB. Mapper mempertahankan kontrak HazardEvent untuk *field* inti, lalu menempatkan *field* di luar daftar yang dikenal ke attributes. Dengan demikian, perubahan aditif seperti confidence_level tidak memerlukan perubahan mapper atau redeploy.

Alternatif mekanisme mapping

Mekanisme	Penilaian
JSON dinamis (dipilih)	Field inti divalidasi, field tambahan masuk ke attributes tanpa perubahan mapper. Dukungan terbatas pada perubahan aditif.
Struct Go dengan field tetap	Mudah diperiksa saat kompilasi, tetapi decoder standar mengabaikan field tambahan. Decoder yang menolak field tak dikenal dapat gagal ketika skema bertambah.
Struct dengan peta ekstensi	Dapat mempertahankan field tambahan, tetapi memerlukan mekanisme untuk menangkapnya secara eksplisit.
Schema registry / Avro / Protobuf	Versi dan kompatibilitas skema lebih terkelola, tetapi membutuhkan dukungan serta koordinasi sumber. Lingkup saat ini memakai API JSON dari dua mock.

Tabel 6. Alternatif mekanisme mapping

Validasi *field* inti

Validasi *field* inti tetap diperlukan agar identitas, jenis bahaya, waktu, dan severity tidak dipetakan dari data yang ambigu. Pemisahan antara *field* inti yang divalidasi dan attributes yang fleksibel menjaga konsistensi kanonik sekaligus mempertahankan informasi tambahan.

5.3 Batasan yang disadari

- Dukungan evolusi skema dibatasi pada penambahan field yang tidak mengubah kontrak field wajib. Field wajib yang hilang atau di-rename, tipe yang tidak dapat dipetakan, timestamp invalid, serta enum baru menyebabkan record ditolak dan dicatat. Kesalahan mapping hanya

mengisolasi record tersebut, loop poller melanjutkan record berikutnya dalam batch. Sebaliknya, kegagalan penyimpanan MongoDB atau publikasi RabbitMQ menghentikan siklus polling saat itu. Field tambahan pada record yang ditolak tidak disimpan ke attributes, dan record invalid belum memiliki mekanisme quarantine atau retry khusus.

- Validasi representasi integer telah diperketat. requireInt hanya menerima float64 yang finite, tidak memiliki bagian desimal, dan berada dalam rentang int platform, NaN, positive/negative infinity, nilai pecahan, serta positive/negative overflow ditolak tanpa truncation. Batasan yang tersisa adalah validasi ini belum menetapkan seluruh batas semantik domain, dan field tambahan seperti confidence_level tetap diteruskan tanpa validasi makna atau rentang khusus.
- Referensi nama dan koordinat gunung masih statis. volcano_id yang tidak dikenal menghasilkan area_name "Unknown Volcano (<id>)" dengan koordinat nol. Record tetap diterima, tetapi kualitas lokasinya perlu dibedakan dari gunung yang ada pada referensi.
- Upsert menggunakan hazard_id deterministik, tetapi source ID mock dapat berulang setelah mock dibuat ulang dan berbenturan dengan data MongoDB yang masih tersimpan. Bukti harus mencocokkan source_ref_id serta occurred_at. Penyimpanan MongoDB dan publikasi RabbitMQ dilakukan terpisah tanpa outbox, retry dapat mengulang publikasi dan memperbarui ingested_at. Mekanisme ini belum menjamin exactly-once atau transaksi atomik antara store dan broker.

5.4 Bukti Penyelesaian

Bukti 1: Skema awal dan mapping kedua sumber

```
{
  "report_id": "pvmbg-000121",
  "volcano_id": "G-001",
  "alert_level": "Normal",
  "eruption_count_24h": 0,
  "ash_column_height_m": 2419.2,
  "reported_at": "2026-10-07T04:28:56.69749804Z"
}

{
  "hazard_id": "hz-pvmbg-000121",
  "source_ref_id": "pvmbg-000121",
  "hazard_type": "VOLCANIC",
  "severity": "NORMAL",
  "area_name": "Gunung Merapi",
  "attributes": {
    "ash_column_height_m": 2419.2,
    "eruption_count_24h": 0
  }
}
```

Gambar 4. Respons PVMBG dan HazardEvent sebelum penambahan field

Respons PVMBG memuat report_id pvmbg-000121 dengan alert_level Normal. Setelah diproses aggregator, report yang sama muncul sebagai hz-pvmbg-000121 bertipe VOLCANIC dengan severity NORMAL dan area_name Gunung Merapi. Nilai ash_column_height_m dan eruption_count_24h tetap berada pada attributes. Pada kedua respons belum terdapat confidence_level.

Bukti skema awal BMKG — sumber dan HazardEvent

source-bmkg-selected.json


```
{
  "event_id": "bmkg-000118",
  "magnitude": 6.8,
  "depth_km": 65.8,
  "epicenter_lat": 1.9735,
  "epicenter_lon": 100.9945,
  "region_name": "Laut Banda",
  "occurred_at":
"2026-10-07T04:28:56.887361137Z",
  "potential_tsunami": false
}
```


api-seismic-selected.json


```
{
  "hazard_id": "hz-bmkg-000118",
  "source_ref_id": "bmkg-000118",
  "source": "BMKG",
  "hazard_type": "SEISMIC",
  "severity": "SIAGA",
  "area_name": "Laut Banda",
  "latitude": 1.9735,
  "longitude": 100.9945,
  "occurred_at": "2026-10-07T04:28:56.887Z",
  "attributes": {
    "depth_km": 65.8,
    "magnitude": 6.8,
    "potential_tsunami": false
  }
}
```


Gambar 5. Respons BMKG dan HazardEvent pada skema awal

Event BMKG bmkg-000118 dipetakan menjadi hz-bmkg-000118 dengan source BMKG, hazard_type SEISMIC, area_name Laut Banda, serta koordinat dan occurred_at yang sama. Nilai magnitude 6,8 menghasilkan severity SIAGA karena tidak ada warning terkait untuk event ini. depth_km, magnitude, dan potential_tsunami tetap tersedia pada attributes. Bersama bukti PVMBG sebelumnya, hasil ini menunjukkan bahwa Aggregator menghasilkan HazardEvent sesuai aturan pemetaan untuk kedua sumber pada skema awal.

Bukti 2: Perubahan skema saat sistem berjalan


```
{
  "report_id": "pvmbg-000123",
  "volcano_id": "G-003",
  "alert_level": "Siaga",
  "eruption_count_24h": 9,
  "ash_column_height_m": 3590.1,
  "reported_at": "2026-10-07T04:29:06.698036956Z",
  "confidence_level": 0.85
}

{
  "hazard_id": "hz-pvmbg-000123",
  "source_ref_id": "pvmbg-000123",
  "hazard_type": "VOLCANIC",
  "severity": "SIAGA",
  "attributes": {
    "ash_column_height_m": 3590.1,
    "confidence_level": 0.85,
    "eruption_count_24h": 9
  }
}
```


Gambar 6. Report PVMBG dan HazardEvent setelah POST /admin/schema-version

Perubahan skema dipicu saat sistem berjalan melalui POST /admin/schema-version dengan body {"enabled": true}, respons schema-on.json menunjukkan enabled bernilai true. Report baru pvmbg-000123 kemudian memuat confidence_level 0.85. Pada respons API, nilai yang sama muncul di attributes untuk hz-pvmbg-000123. Field kanonik tetap terbentuk dan record lama pvmbg-000121 tetap tidak memiliki confidence_level. Tidak ada perubahan kode mapper maupun intervensi manual selama pengujian.

container-before.json

```
pvmbg-mock
container_id: 048adb8f62cd06caf2bc78030eb054f28c1bafcf21cbad95f90302bfb81e0340
started_at: 2026-10-07T04:12:21.522402497Z
restart_count: 0

aggregator
container_id: f0f6f628c26dd1d2c7f99e407e77bb22162634dfcb4c62371e8001e1ffa651dd
started_at: 2026-10-07T04:13:15.809338284Z
restart_count: 0
```

container-after.json

```
pvmbg-mock
container_id: 048adb8f62cd06caf2bc78030eb054f28c1bafcf21cbad95f90302bfb81e0340
started_at: 2026-10-07T04:12:21.522402497Z
restart_count: 0

aggregator
container_id: f0f6f628c26dd1d2c7f99e407e77bb22162634dfcb4c62371e8001e1ffa651dd
started_at: 2026-10-07T04:13:15.809338284Z
restart_count: 0
```

Gambar 7. Identitas container sebelum dan sesudah perubahan skema

ID container, started_at, dan restart_count pada PVMBG serta aggregator sama sebelum dan sesudah perubahan. Restart count keduanya tetap 0. Ini menunjukkan bahwa report dengan field baru diterima tanpa restart kedua service pada pengujian tersebut.

Bukti 3: Penyimpanan field baru dan penelusuran correlation ID

mongo-volcanic-schema-on.json

```
{
  "hazard_id": "hz-pvmbg-000123",
  "source_ref_id": "pvmbg-000123",
  "occurred_at": "2026-10-07T04:29:06.698Z",
  "ingested_at": "2026-10-07T04:29:11.171Z",
  "attributes": {
    "ash_column_height_m": 3590.1,
    "confidence_level": 0.85,
    "eruption_count_24h": 9
  }
}
```

correlation-logs.txt — event hz-pvmbg-000123

```
{
  "time": "2026-10-07T04:29:11.177757497Z",
  "level": "INFO",
  "msg": "hazard event ingested",
  "service": "aggregator",
  "hazard_id": "hz-pvmbg-000123",
  "hazard_type": "VOLCANIC",
  "severity": "SIAGA",
  "correlation_id": "1aaf393227400fb0b506ead342eee1a1"
}
```

Gambar 8. Dokumen MongoDB dan log ingest untuk report yang sama

Dokumen MongoDB menyimpan confidence_level 0.85 pada attributes milik hz-pvmbg-000123, sesuai dengan respons sumber dan API. Log aggregator mencatat hazard event ingested untuk hazard_id yang sama dengan correlation_id 1aaf393227400fb0b506ead342eee1a1. Ini menunjukkan bahwa field tambahan tetap tersimpan dan event dapat ditelusuri pada proses ingest.

Timestamp occurred_at mengikuti waktu report sumber, sedangkan ingested_at menunjukkan waktu penyimpanan. Perbedaan presisi nanodetik pada sumber dan milidetik pada MongoDB diperhitungkan saat mencocokkan data.

Bukti 4: Konsistensi kebijakan field tambahan

mapper-tests.txt — hasil test

```
--- PASS: TestMapSeismicEventPreservesUnknownFields (0.00s)
--- PASS: TestMapSeismicEventRejectsInvalidInput (0.00s)
--- PASS: TestMapSeismicEventMagnitudeBoundaries (0.00s)
--- PASS: TestMapSeismicEventWarningOverridesMagnitude (0.00s)
--- PASS: TestMapVolcanicReportPreservesUnknownFields (0.00s)
--- PASS: TestMapVolcanicReportRejectsInvalidInput (0.00s)
--- PASS: TestMapVolcanicReportKnownAndUnknownVolcano (0.00s)
--- PASS: TestMapVolcanicReportFractionalEruptionCountCurrentBehavior (0.00s)
PASS
ok      aggregator/internal/mapper      0.873s
```

Gambar 9. Hasil tes mapper untuk field tambahan dan input invalid

Keluaran tes menunjukkan PASS untuk penerusan field tambahan yang tidak dikenal, termasuk nilai primitif, object, dan array. Tes mapper juga membuktikan bahwa nilai integer valid tetap diterima, sedangkan eruption_count pecahan (3.7), NaN, positive/negative infinity, serta positive/negative overflow ditolak tanpa truncation. Seluruh test Aggregator dan go vet ./... lulus. Hasil tersebut konsisten dengan kebijakan bahwa field tambahan diteruskan ke attributes selama kontrak field wajib tetap valid, sedangkan perubahan tipe atau nilai yang tidak valid pada field inti menyebabkan record ditolak.

5.5 Jawaban Pertanyaan

Mekanisme dan perubahan skema yang didukung

Mekanisme yang dipilih adalah dynamic field mapping pada JSON dengan kebijakan additive-only. Sumber dibaca sebagai map[string]any, field inti divalidasi dan dipetakan ke HazardEvent, sedangkan field tambahan diteruskan ke attributes. Penambahan confidence_level terbukti sampai API dan MongoDB tanpa perubahan mapper, restart, atau redeploy.

Jenis perubahan yang didukung dan tidak didukung

Jenis perubahan skema	Status	Perilaku sistem
Field baru ditambahkan (aditif)	Didukung	Nilai JSON top-level, termasuk primitif, object, dan array, diteruskan ke attributes.
Field wajib hilang	Tidak didukung	Record ditolak dan galat dicatat.
Tipe field wajib berubah	Tidak didukung	Record ditolak jika nilainya tidak dapat dipetakan ke tipe yang diharapkan.
Field di-rename	Tidak didukung	Field wajib lama dianggap hilang, record ditolak dan nama baru tidak disimpan sebagai attributes parsial.
Nilai enum baru	Tidak didukung	Record ditolak dan galat dicatat.

Tabel 7. Jenis perubahan skema yang didukung dan tidak didukung

Kebijakan field tak dikenal

Kebijakan yang dideklarasikan adalah meneruskan field tak dikenal ke attributes. Kebijakan ini berlaku konsisten untuk kedua sumber: field tambahan SeismicEvent dan data TsunamiWarning disimpan dengan prefiks tsunami_warning_, sedangkan field tambahan VolcanicReport seperti confidence_level dipertahankan apa adanya. Record lama tidak diubah secara retroaktif dan record invalid tidak menghasilkan attributes parsial.

BAB 6

Problem 2: *Concurrency, Availability, and Graceful Degradation*

6.1 Rancangan

Masalah

Gateway memanggil BMKG dan PVMBG secara berurutan sehingga sumber yang lambat menahan sumber lain (head-of-line blocking). Permintaan client yang tidak dibatasi dapat menghabiskan koneksi, memori, dan kapasitas pemrosesan (resource exhaustion). Kegagalan satu sumber juga dapat membuat seluruh respons gagal, meskipun data dari sumber lain masih tersedia. Sistem perlu memisahkan pekerjaan tiap sumber, membatasi beban request, dan tetap menyajikan data yang tersedia dengan penanda degradasi yang jelas.

Tiga kegagalan dan padanan solusinya

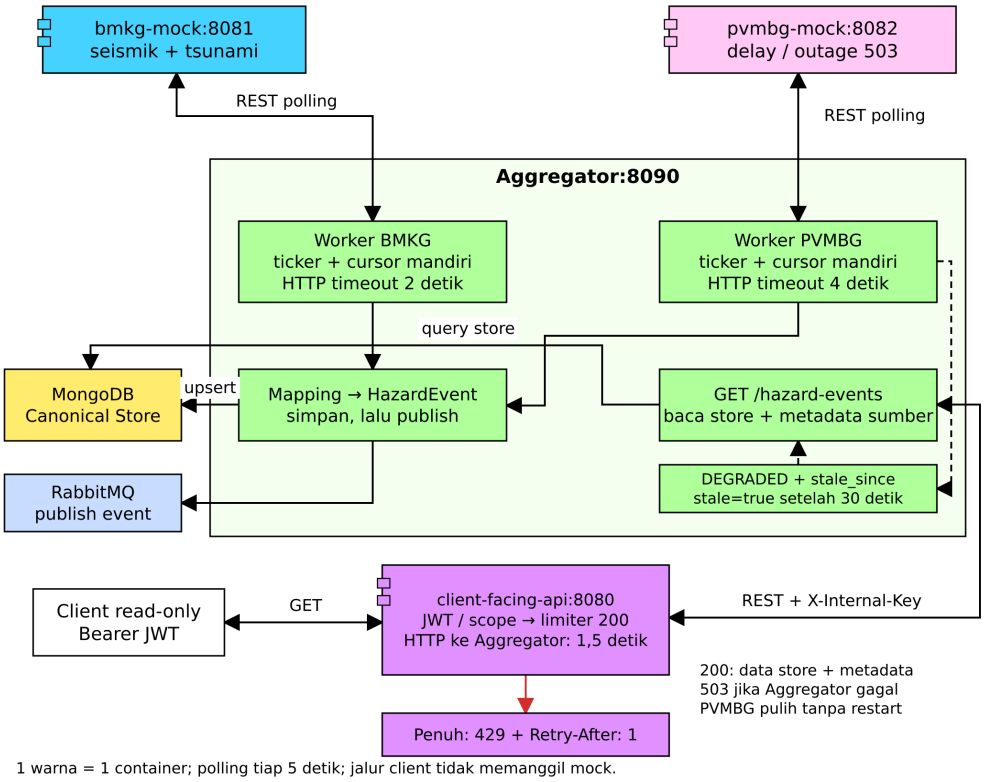
Kegagalan	Penyebab pada desain lama	Solusi pada sistem ini
Head-of-line blocking	BMKG lalu PVMBG dipanggil berurutan, sumber lambat menahan sumber cepat.	Worker polling terpisah per sumber. Jalur baca juga dipisahkan dari ingest: client membaca MongoDB melalui Aggregator dan tidak memanggil mock.
Resource exhaustion	Tidak ada batas permintaan yang diproses bersamaan.	Limiter inflight pada client-facing-api, connection pool terukur, timeout di setiap sisi, batas parameter, dan batas body respons.
Tanpa degradasi terkendali	Satu sumber mati menyebabkan seluruh respons gagal.	Data terakhir tetap disajikan dari Canonical Store dengan source_status, stale_since, dan stale, kegagalan Aggregator menjadi 503 eksplisit.

Tabel 8. Kegagalan Problem 2 dan solusinya

Model konkurensi: pemisahan ingest dan serve

Aggregator menjalankan dua goroutine polling mandiri melalui Run, runBMKGWorker, dan runPVMBGWorker pada poller.go. Masing-masing memiliki ticker serta cursor sendiri, yaitu bmkgSince dan pvmbgSince. Ketika PVMBG memerlukan 3 detik atau mencapai timeout 4 detik, worker BMKG tetap berjalan pada jadwalnya sendiri setiap 5 detik. Pemisahan ini menerapkan pola bulkhead secara logis: kegagalan satu sumber tidak langsung menghentikan worker sumber lain.

Jalur baca tidak bergantung pada mock. Permintaan client mengalir melalui client-facing-api ke API Aggregator, lalu dibaca dari MongoDB tanpa panggilan ke BMKG atau PVMBG. Karena itu, permintaan BMKG-only dengan hazard_type=SEISMIC secara struktural tidak menunggu PVMBG, latensinya berasal dari client-facing-api, Aggregator, dan MongoDB.



Gambar 10. Model konkurensi: ingest di latar belakang, serve dari store

Mitigasi risiko resource exhaustion

Mekanisme	Nilai / perilaku
Limiter inflight (httpx.Limit)	Kapasitas CLIENT_API_MAX_INFLIGHT, default 200. Saat penuh, request langsung dijawab 429 dengan Retry-After: 1 dan tidak dimasukkan ke antrean.
Urutan middleware	Autentikasi dan pemeriksaan scope berjalan sebelum limiter sehingga request tanpa token atau scope sah tidak merebut slot.
Connection pool ke Aggregator	MaxIdleConns, MaxIdleConnsPerHost, dan MaxConnsPerHost disamakan dengan limiter. IdleConnTimeout 90 detik dan ResponseHeaderTimeout sama dengan timeout Aggregator.
Timeout klien HTTP	client-facing-api ke Aggregator 1,5 detik, Aggregator ke BMKG 2 detik dan ke PVMBG 4 detik, operasi MongoDB 5 detik.
Timeout server HTTP	ReadHeaderTimeout 5 detik, ReadTimeout 10 detik, WriteTimeout 15 detik, dan IdleTimeout 60 detik pada client-facing-api serta Aggregator.
Batas parameter limit	Maksimal 200 pada client-facing-api dan 500 pada Aggregator.
Batas body respons upstream	Respons Aggregator yang dibaca client-facing-api dibatasi 8 MB.

Tabel 9. Mekanisme mitigasi resource exhaustion

Degradasi terkendali saat outage PVMBG

1. Mock PVMBG mengembalikan HTTP 503 segera setelah POST /admin/outage dengan body {"enabled": true}, request tidak dibiarkan menggantung.

2. pvmbgclient mengenali respons tersebut sebagai ErrOutage. Poller mencatat bahwa PVMBG sedang outage, melewati siklus itu tanpa memengaruhi data seismik, mengubah source_status menjadi DEGRADED, dan mengisi stale_since.
3. Worker BMKG tetap berjalan karena berada pada goroutine terpisah, sehingga data seismik baru tetap di-ingest dan permintaan SEISMIC tetap dilayani.
4. Permintaan VOLCANIC tetap memperoleh HTTP 200 dari data terakhir di MongoDB beserta meta.sources.pvmbg yang memuat source_status, last_success_at, stale_since, dan stale. Nilai stale menjadi true setelah AGGREGATOR_PVMBG_STALE_AFTER_SECONDS, yaitu 30 detik sejak stale_since.
5. Jika Aggregator sendiri tidak terjangkau, client-facing-api mengembalikan HTTP 503 upstream_unavailable dengan Retry-After, bukan HTTP 500 atau timeout tanpa penjelasan.
6. Setelah outage dimatikan, polling berikutnya yang berhasil mengembalikan status menjadi HEALTHY, menghapus stale_since, dan menerima report baru tanpa restart service.

6.2 Alasan dan Alternatif

Alasan memisahkan ingest dan serve

Tiga alternatif dipertimbangkan. Pertama, pemanggilan sekuensial BMKG lalu PVMBG pada setiap request client, yaitu desain lama yang menimbulkan head-of-line blocking. Kedua, fan-out paralel ke kedua sumber pada setiap request, cara ini mengurangi ketergantungan urutan, tetapi respons gabungan tetap menunggu sumber yang terlambat dan setiap request client menambah beban ke sumber. Ketiga, pemisahan ingest dan serve yang dipilih: data diambil melalui worker latar belakang dan permintaan client dilayani dari Canonical Store.

Alternatif model konkurensi

Model	Penilaian
Pemanggilan sekuensial per request	Sumber lambat menahan seluruh respons dan beban client diteruskan ke sumber.
Fan-out paralel per request	Mengurangi ketergantungan urutan, tetapi respons gabungan tetap dapat menunggu sumber terlambat. Beban client juga meningkatkan beban sumber.
Ingest dan serve terpisah (dipilih)	Polling latar belakang membatasi frekuensi akses sumber, client membaca store. Konsekuensinya, data adalah snapshot dan kesegarannya bergantung pada polling serta retry.

Tabel 10. Alternatif model konkurensi

Konsekuensi terhadap kesegaran data

Trade-off pilihan ini adalah kesegaran data. Dalam kondisi normal, snapshot dapat tertinggal setidaknya hingga satu siklus polling 5 detik, tetapi interval tersebut bukan batas maksimum keterlambatan karena latensi sumber, pemrosesan batch, retry, dan outage dapat memperpanjang jeda. Saat sumber mati, data terakhir tetap disajikan dan diberi penanda stale_since serta stale. Untuk koordinasi bencana, data lama yang diberi penanda eksplisit dinilai lebih berguna daripada seluruh respons gagal.

Penolakan saat kapasitas penuh

Load shedding melalui HTTP 429 dipilih daripada antrean panjang. Request yang tidak memperoleh slot langsung menerima Retry-After: 1 sehingga tidak ikut menumpuk waktu tunggu dan tidak meneruskan pekerjaan baru ke Aggregator. Respons 429 diperlakukan sebagai penolakan terkendali dan dilaporkan terpisah dari kegagalan non-429.

6.3 Batasan yang disadari

- Isolasi *worker* bersifat logis, bukan isolasi sumber daya penuh. BMKG dan PVMBG masih menggunakan proses aggregator, MongoDB, dan publisher yang sama. Gangguan pada store, broker, CPU, atau memori dapat memengaruhi keduanya. Pengambilan seismic dan tsunami warning tetap berurutan di *worker* BMKG, kegagalan warning membuat siklus BMKG dilewati untuk menghindari severity yang salah.
- Limiter 200 berlaku pada handler client-facing-api setelah autentikasi dan pemeriksaan scope. Mekanisme ini tidak membatasi seluruh koneksi masuk, biaya verifikasi token, atau seluruh resource pada layanan lain. Connection pool dan timeout membantu membatasi beban, tetapi tidak dengan sendirinya menjamin p95 di bawah 300 ms. Karena itu, mekanisme tersebut diperlakukan sebagai mitigasi risiko resource exhaustion, bukan sebagai bukti bahwa tekanan resource tidak dapat terjadi. Respons 429 harus dilaporkan terpisah dari respons berhasil dan kegagalan lain.
- Status PVMBG mencerminkan keberhasilan fetch HTTP. Poller menandai HEALTHY sebelum seluruh *record* selesai dipetakan dan disimpan, sehingga status sehat bukan bukti bahwa setiap *record* berhasil di-*ingest*. stale=true dihitung setelah 30 detik sejak kegagalan fetch pertama, ini bukan ukuran umur setiap *event*. Metadata yang tersedia saat ini khusus PVMBG, sehingga kegagalan BMKG tidak memiliki penanda sumber yang setara.
- Cursor berada di memori dan hilang saat restart, *upsert* mengurangi duplikasi dokumen, tetapi *retry* publikasi masih mungkin. Ketika MongoDB tidak tersedia, jalur baca tidak dapat menyajikan cache data di luar store. Graceful degradation terhadap *outage* PVMBG tidak berarti seluruh dependensi dapat gagal tanpa dampak.
- TTL access token default adalah 60 detik, sedangkan skrip k6 tidak melakukan refresh. Pada pengujian, TTL efektif disetel 300 detik dan token baru diambil sebelum setiap skenario. Perlindungan limiter belum mencakup seluruh biaya autentikasi, validitas token tetap perlu diperhitungkan pada pengujian berikutnya.

6.4 Bukti Penyelesaian

Bukti 1: Sustained load dengan 50 VU selama 60 detik

sustained-console.txt — hasil akhir k6

```
    ✓ 'rate>0.99' rate=100.00%
    ✓ 'rate<0.01' rate=0.00%
    checks_total.......: 53008   883.042088/s
    checks_succeeded...: 100.00% 53008 out of 53008
    checks_failed......: 0.00%   0 out of 53008
    controlled_429s.................: 0      0/s
    data_requests...................: 53008  883.042088/s
    non_429_failures................: 0.00%  0 out of 53008
    seismic_status_2xx..............: 26507  441.57102/s
    volcanic_status_2xx.............: 26501  441.471068/s
    http_req_duration...............: avg=56.32ms med=51.14ms p(90)=94.74ms
p(95)=110.71ms p(99)=146.97ms max=246.7ms
    http_req_failed.................: 0.00%  0 out of 53008
    vus.............................: 50     min=50         max=50
sustained_parallel_load ✓ [ 100% ] 50 VUs  1m0s
```

sustained-run.json

```
{
  "started_utc": "2026-10-06T13:48:20.9544485Z",
  "ended_utc": "2026-10-06T13:49:21.4440108Z",
  "exit_code": 0,
  "vus": 50,
  "duration": "60s",
  "token_expires_in": 300
}
```

Gambar 11. Hasil k6 pada beban 50 VU selama 60 detik

Keluaran k6 menunjukkan 53.008 request data dengan throughput 883,042 request/detik. Seluruh request data memperoleh respons 2xx dan seluruh check berhasil. Nilai p50 sebesar 51,148 ms, p95 sebesar 110,715 ms, dan p99 sebesar 146,979 ms berasal dari summary final. Selama measurement window tidak ditemukan respons 401, 429, 500, 503, maupun network error, dan pengujian selesai dengan exit code 0. Kombinasi hasil tersebut menunjukkan bahwa layanan tetap tersedia sepanjang measurement window pada beban yang diuji.

Pengujian memakai limiter 200, timeout API ke Aggregator 1,5 detik, interval polling 5 detik, dan token baru dengan TTL 300 detik. Pengujian menjalankan 50 VU secara konstan selama 60 detik. Pemantauan menunjukkan 50 koneksi API berstatus established pada 11 sampel dalam rentang 54,06 detik.

Bukti 2: BMKG tetap berjalan saat PVMBG lambat

isolation-console.txt — threshold dan metrik SEISMIC

```
    ✓ 'p(95)<300' p(95)=40.68ms
    ✓ 'count>=4' count=6
    checks_succeeded...: 100.00% 53192 out of 53192
    checks_failed......: 0.00%   0 out of 53192
    data_requests.................: 53132   874.425112/s
    seismic_duration..............: avg=23.25ms med=21.56ms p(90)=35.25ms
p(95)=40.68ms p(99)=54.06ms max=119.95ms
    seismic_monitor_success.......: 100.00% 60 out of 60
    seismic_status_2xx............: 25517   419.948535/s
    seismic_updates...............: 6       0.098746/s
seismic_only             ✓ [ 100% ] 10 VUs  1m0s
volcanic                 ✓ [ 100% ] 10 VUs  1m0s
```

isolation-run.json

```
{
  "started_utc": "2026-10-06T13:51:20.8533640Z",
  "ended_utc": "2026-10-06T13:52:22.0534343Z",
  "exit_code": 0,
  "seismic_vus": 10,
  "volcanic_vus": 10,
  "freshness_vus": 1,
  "duration": "60s",
  "token_expires_in": 300
}
```

Gambar 12. Latensi dan pembaruan SEISMIC saat delay PVMBG 3000 ms

PVMBG diberi delay tetap 3000 ms, kemudian pengujian dijalankan dengan 10 VU SEISMIC, 10 VU VOLCANIC, dan 1 VU monitor selama 60 detik. Keluaran menunjukkan p95 SEISMIC 40,68 ms, di bawah target 300 ms, dengan p50 21,56 ms dan p99 54,07 ms. Sebanyak 25.517 request SEISMIC menghasilkan 2xx dan monitor berhasil pada seluruh 60 pemeriksaan.

Monitor mencatat enam pembaruan seismic, melebihi target empat pembaruan. Seluruh 53.132 request data menghasilkan 2xx dengan throughput 874,43 request/detik dan exit code 0. Ini menunjukkan bahwa worker BMKG tetap mengambil data baru dan API tetap melayani pembacaan saat sumber PVMBG lambat. Latensi yang digunakan adalah latensi kelompok SEISMIC, bukan gabungan request data dan monitor.

Bukti 3: Degradasi dan pemulihan saat outage PVMBG

healthy.json — field respons terpilih


```
{
  "observed_at": "2026-10-06T13:58:07.6664091Z",
  "status": 200,
  "hazard_id": "hz-pvmbg-000052",
  "pvmbg": {
    "source_status": "HEALTHY",
    "last_success_at": "2026-10-06T13:58:04.466219842Z",
    "stale": false
  }
}
```


degraded.json — field respons terpilih


```
{
  "observed_at": "2026-10-06T13:58:08.8018197Z",
  "status": 200,
  "hazard_id": "hz-pvmbg-000052",
  "pvmbg": {
    "source_status": "DEGRADED",
    "last_success_at": "2026-10-06T13:58:04.466219842Z",
    "stale_since": "2026-10-06T13:58:08.382979581Z",
    "stale": false
  }
}
```


Gambar 13. Respons sebelum outage dan setelah PVMBG mengalami kegagalan

Sebelum outage, metadata PVMBG menunjukkan HEALTHY dan stale=false. Setelah fetch gagal, status berubah menjadi DEGRADED dan stale_since terisi. API tetap mengembalikan HTTP 200 dengan data volcanic terakhir, hz-pvmbg-000052. Data seismic juga terus bertambah selama sumber PVMBG tidak tersedia.

stale.json — field respons terpilih


```
{
  "observed_at": "2026-10-06T13:58:40.0375800Z",
  "status": 200,
  "hazard_id": "hz-pvmbg-000052",
  "pvmbg": {
    "source_status": "DEGRADED",
    "last_success_at": "2026-10-06T13:58:04.466219842Z",
    "stale_since": "2026-10-06T13:58:08.382979581Z",
    "stale": true
  }
}
```


recovered.json — field respons terpilih


```
{
  "observed_at": "2026-10-06T13:58:44.2058779Z",
  "status": 200,
  "hazard_id": "hz-pvmbg-000056",
  "pvmbg": {
    "source_status": "HEALTHY",
    "last_success_at": "2026-10-06T13:58:44.132597376Z",
    "stale": false
  }
}
```


Gambar 14. Penanda stale dan respons setelah PVMBG pulih

Pada pengamatan sekitar 31,65 detik setelah stale_since, stale berubah menjadi true. Setelah outage dimatikan, status kembali HEALTHY, stale=false, dan stale_since tidak ada. Nilai last_success_at bertambah dan report baru hz-pvmbg-000056 muncul. Identitas container serta waktu mulai PVMBG dan aggregator tetap sama selama outage dan pemulihan.

Skenario trafik outage dengan 10 VU selama 30 detik menghasilkan 21.698 request data, seluruhnya 2xx, serta 43.396 check berhasil dengan exit code 0. p95 SEISMIC sebesar 23.20 ms dan VOLCANIC sebesar 22,06 ms. Pemeriksaan stale dilakukan satu interval setelah batas 30 detik agar tidak membaca tepat pada saat transisi.

Bukti 4: Penolakan 429 dan respons 503

limiter-and-503-tests.txt — potongan hasil tes

```
=== RUN   TestLimitRejectsOverflowAndReleasesSlots
--- PASS: TestLimitRejectsOverflowAndReleasesSlots (0.00s)
=== RUN   TestLimitRejectsOverflowAndReleasesSlots
--- PASS: TestLimitRejectsOverflowAndReleasesSlots (0.00s)
... (keluaran berulang dipotong) ...
PASS
ok      client-facing-api/internal/httpx        0.524s
--- PASS: TestRoutesAuthenticateAndAuthorizeBeforeLimiter (0.00s)
--- PASS: TestGetHazardEventsReturnsServiceUnavailableForUpstreamFailure
(0.00s)
```

limiter-and-503-tests-run.json

```
{
  "exit_code": 0,
  "command": "go test -count=20 -v ./internal/httpx ./internal/api"
}
```

Gambar 15. Hasil tes limiter, autentikasi, dan kegagalan upstream

Tes limiter menggunakan blocking handler berkapasitas dua. Request yang melebihi kapasitas memperoleh HTTP 429 dengan Retry-After: 1, dan slot kembali tersedia setelah request selesai. Tes autentikasi sebelum limiter serta penanganan kegagalan upstream juga menghasilkan PASS. Pengujian diulang 20 kali dengan exit code 0. Bukti 429 ini berasal dari focused Go test yang mengondisikan limiter penuh secara deterministik, bukan dari overload end-to-end pada konfigurasi utama. Pada sustained load dengan kapasitas 200 tidak ditemukan respons 429.

503/headers.txt

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
Retry-After: 2
X-Correlation-Id: p2-503-explicit
Date: Tue, 06 Oct 2026 13:56:17 GMT
Content-Length: 87
```

503/body.json

```
{
  "error": "upstream_unavailable",
  "message": "hazard data source did not answer in time"
}
```

503/metrics.txt

```
status=503;elapsed_seconds=1.506770
```

Gambar 16. Respons API ketika Aggregator tidak dapat menjawab

Saat aggregator dihentikan sementara, client-facing-api mengembalikan HTTP 503 dengan error upstream_unavailable dan Retry-After: 2. Waktu respons tercatat 1506,770 ms, sesuai timeout sekitar 1,5 detik. Aggregator kemudian dipulihkan. Penghentian ini merupakan skenario terpisah dari outage PVMBG, sehingga tidak termasuk pengujian pemulihan tanpa restart di atas.

Hasil load test

Skenario	Beban dan durasi	Volume dan throughput	Latensi dan reliabilitas
p2-sustained	50 VU konstan, 60 detik	53.008 request data, 883,04 req/s	p50 51,15 ms, p95 110,72 ms, p99 146,98 ms, error non-429 0%, 429 = 0, seluruh check lulus.
p2-isolation	10 VU SEISMIC + 10 VU VOLCANIC + 1 VU monitor, 60 detik, PVMBG delay 3 detik	53.132 request data, 874,43 req/s, 25.517 request SEISMIC	SEISMIC p50 21,56 ms, p95 40,68 ms, p99 54,07 ms, enam pembaruan, error non-429 0%, 429 = 0.

Tabel 11. Hasil load test Problem 2

6.5 Jawaban Pertanyaan

Model konkurensi dan pencegahan kegagalan

Model yang dipilih adalah dua worker polling mandiri untuk ingest, jalur serve yang membaca canonical store, dan limiter inflight pada client-facing-api. Pemisahan worker mengurangi head-of-line blocking antar sumber. Timeout dan pool koneksi membatasi waktu tunggu, sedangkan limiter memberikan penolakan terkendali saat kapasitas handler terpenuhi.

Saat PVMBG gagal, API menyajikan data terakhir beserta metadata DEGRADED dan stale. Saat aggregator tidak dapat menjawab, API mengembalikan 503 eksplisit. Hasil isolasi dan outage pada Bagian 6.4 mendukung perilaku tersebut, dengan batas isolasi sumber daya bersama sebagaimana dijelaskan pada Bagian 6.3.

Konfigurasi dan tooling load test

Pengujian menggunakan k6 dengan skrip pada tests/k6/. Sustained menggunakan 50 VU selama 60 detik, isolation menggunakan 21 VU termasuk monitor dengan delay PVMBG 3000 ms, outage menggunakan 10 VU selama 30 detik.

Parameter	Nilai
Tool	k6 melalui skrip pada tests/k6/. hey dapat dipakai untuk uji cepat, tetapi bukan sumber angka evidence utama.
Sustained	50 VU konstan selama 60 detik melalui p2-sustained.js.
Isolation	10 VU SEISMIC, 10 VU VOLCANIC, dan 1 VU monitor selama 60 detik melalui p2-isolation.js.
Outage	10 VU selama 30 detik melalui p2-outage.js, ditambah pemeriksaan stale dan recovery.
Endpoint	GET /hazard-events pada client-facing-api dengan hazard_type=SEISMIC atau VOLCANIC, API_BASE_URL default http://localhost:8080.

Token	API_TOKEN dari access token baru. AUTH_ACCESS_TTL_SECONDS disetel 300 karena skrip tidak melakukan refresh token.
Delay PVMBG	PVMBG_DELAY_MIN_MS=3000 dan PVMBG_DELAY_MAX_MS=3000 untuk skenario isolation.
Konfigurasi inti	CLIENT_API_MAX_INFLIGHT=200, timeout API ke Aggregator 1.500 ms, polling 5 detik, stale threshold 30 detik.

Tabel 12. Konfigurasi load testing

Skrip k6 pada repositori

Skrip	Isi	Threshold
p2-sustained.js	50 VU konstan selama 60 detik, traffic SEISMIC dan VOLCANIC bergantian, 429 dihitung sebagai controlled_429s.	non_429_failures < 1%, checks > 99%.
p2-isolation.js	10 VU SEISMIC + 10 VU VOLCANIC dan 1 monitor kesegaran, PVMBG delay 3 detik, memeriksa event seismik baru.	p95 SEISMIC < 300 ms, seismic_updates ≥ 4.
p2-outage.js	Mengaktifkan outage pada setup, memeriksa SEISMIC dan VOLCANIC, lalu mematikan outage pada teardown. Membutuhkan PVMBG_TOKEN.	p95 SEISMIC < 300 ms, VOLCANIC harus 200 dengan metadata DEGRADED/stale dan record vulkanik terakhir tetap tersedia.

Tabel 13. Skrip k6 pada repositori

TTL access token efektif disetel 300 detik dan setiap skenario menggunakan token baru. Konfigurasi ini menjaga token tetap valid selama setup dan pengujian, perubahan TTL tidak memperpanjang token yang sudah diterbitkan.

Definisi dan kesimpulan pengukuran

Throughput yang dilaporkan menggunakan rate metrik data_requests dari k6, yaitu 883.04 request/detik pada sustained dan 874.43 request/detik pada isolation. Request setup, administrasi, serta monitor tidak dicampurkan ke jumlah request data. Latensi SEISMIC pada isolation dilaporkan terpisah agar hasil benar-benar mengukur dampak PVMBG lambat terhadap BMKG.

non_429_failures menghitung kegagalan selain 429 dibagi seluruh request. Pada sustained, jumlah 429 dan kegagalan lain sama-sama 0, sehingga tingkat kegagalan baik terhadap semua request maupun request non-429 adalah 0%. Seluruh check berhasil. Hasil memenuhi target pada beban yang diuji, tetapi belum membuktikan kapasitas maksimum atau 50 VU paralel selama 60 detik selama 60 detik.

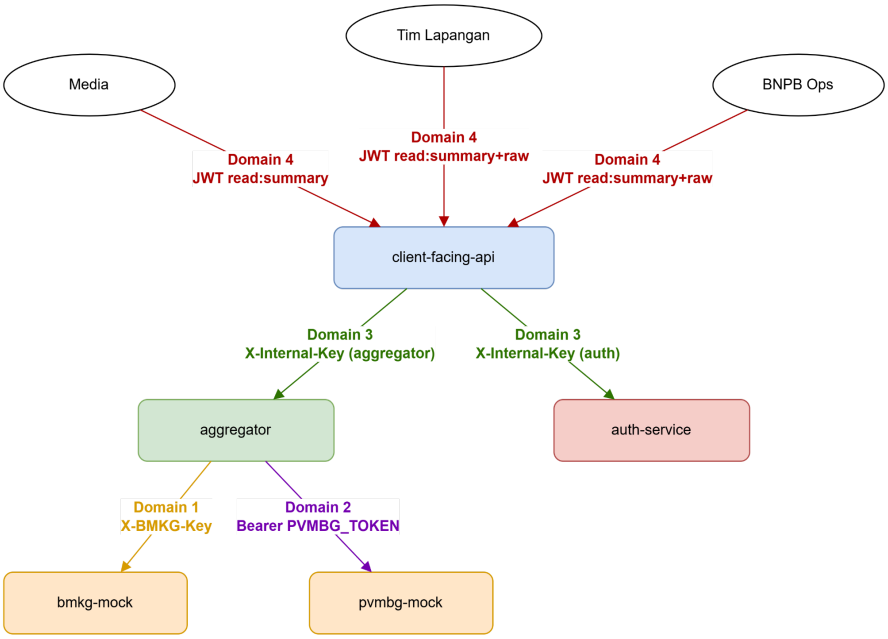
BAB 7

Problem 3: Service to Service Authentication and Trust

7.1 Rancangan

Masalah

Beberapa service harus saling percaya tanpa membagikan satu rahasia bersama, dan tanpa membuat setiap pemeriksaan harus menelepon balik ke pihak yang menerbitkan kredensial setiap kali ada request (yang akan membuatnya jadi titik tunggal kegagalan untuk tiap request data). Sistem ini menerapkan empat domain trust dengan kredensial masing-masing yang tidak saling berlaku, jadi kebocoran satu domain tidak membuka domain lain.



Gambar 17. Empat domain trust dengan kredensial terpisah

Tabel 14. Domain trust dan kredensialnya

Domain	Kredensial	Dipakai oleh
BMKG	Header X-BMKG-Key	aggregator ke bmkg-mock
PVMBG	Header Authorization: Bearer PVMBG_TOKEN	aggregator ke pvmbg-mock
Internal BNPB	X-Internal-Key (nilai berbeda untuk aggregator dan untuk auth-service)	antar service BNPB
Client downstream	JWT dari auth-service	Media, Tim Lapangan, BNPB Ops ke client-facing-api

Kedua mock menolak kredensial yang salah atau tertukar antar domain. Respons 401 diberikan bila skema header berbeda atau tidak ada (misalnya token PVMBG dikirim ke BMKG lewat Authorization), sedangkan 403 diberikan bila skema header benar tetapi nilainya salah atau milik domain lain (misalnya X-BMKG-Key berisi nilai token PVMBG).

Penandatanganan asimetris

Untuk domain client downstream, auth-service menandatangani token memakai EdDSA (Ed25519). auth-service memegang kunci privat, client-facing-api hanya memegang kunci publik. Kunci publik bisa didapat lewat dua jalur, kalau variabel AUTH_PUBLIC_KEY diisi, client-facing-api memakainya langsung tanpa memanggil auth-service sama sekali (key pinning), kalau kosong, ia mengambilnya lewat GET /public-key saat start dengan retry 30 kali setiap 2 detik. Jalur verifikasi dikunci hanya menerima algoritma EdDSA lewat jwt.WithValidMethods, sehingga serangan alg-confusion atau alg=none tidak dapat dipakai memalsukan token.

Tiga identitas, scope ditetapkan di server

auth-service membaca tiga pasang client_id dan client_secret dari environment saat start (untuk Media, Tim Lapangan, dan BNPB Ops), dan menolak untuk start sama sekali kalau ada ID atau secret yang sama di antara ketiganya. Scope tiap klien ditetapkan tetap di konfigurasi server, bukan dikirim oleh client saat login, sehingga hak akses melekat pada identitas yang diverifikasi dan tidak bisa dinaikkan sendiri oleh client.

Tabel 15. Identitas client dan scope

Client	Scope	Field yang boleh
Media / Pers	read:summary	7 field Ringkasan
Tim Lapangan	read:summary read:raw	11 field (Ringkasan + source_ref_id, latitude, longitude, attributes)
BNPB Ops	read:summary read:raw	11 field

Mekanisme token

Satu endpoint POST /token dengan dua jenis grant.

Tabel 16. Jenis grant pada endpoint POST /token

grant_type	Fungsi	Hasil
client_credentials	Login awal	Access token (TTL 60 detik) dan refresh token (TTL 600 detik)

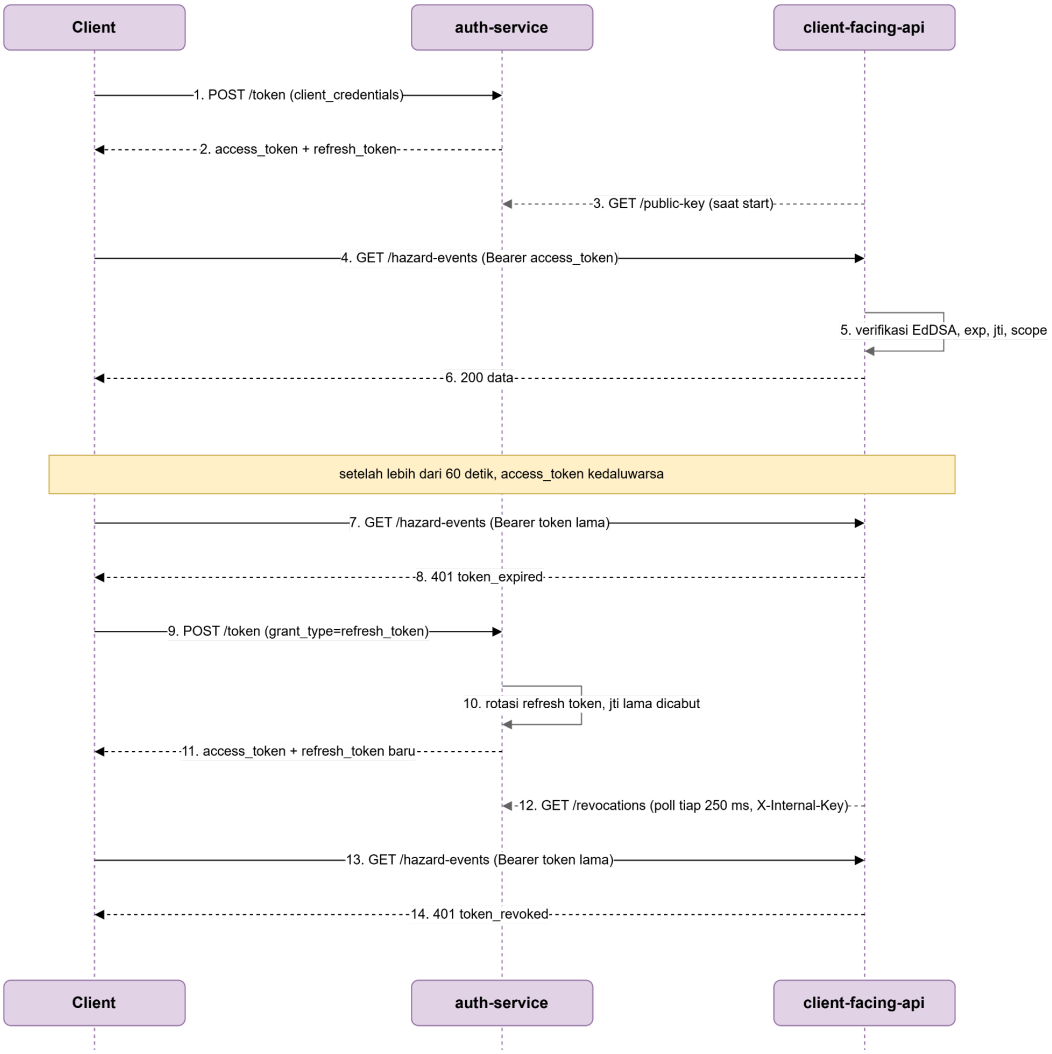
refresh_token	Perpanjang sesi tanpa login ulang	Access token baru, refresh token lama dirotasi jadi baru

Permintaan dikirim form-encoded (bukan JSON), client secret dibaca lewat PostForm saja, tidak pernah lewat query string, agar secret tidak tercatat di log akses. Secret dibandingkan lewat hash SHA-256 dengan constant-time comparison, agar tahan timing attack. Response /token selalu disertai header Cache-Control: no-store dan Pragma: no-cache, agar token tidak ikut tersimpan di cache perantara.

Pencabutan dan deteksi pencurian

Setiap access token punya jti. Saat refresh terjadi, access token lama dicabut lewat jti-nya dan refresh token lama ditandai tidak valid. Refresh token tidak pernah disimpan dalam bentuk mentah di memori auth-service, hanya hash-nya. client-facing-api menyinkronkan daftar token yang dicabut lewat polling ke GET /revocations setiap CLIENT_API_REVOCATION_POLL_MS, dilindungi header X-Internal-Key.

Jika sebuah refresh token yang sudah dipakai dicoba dipakai lagi, sistem menganggap itu tanda pencurian dan mencabut seluruh sesi terkait, bukan hanya refresh token tersebut.



Gambar 18. Alur login, penggunaan, kedaluwarsa, dan refresh token

Proyeksi field berbasis scope

client-facing-api memotong field lewat allow-list, Ringkasan (7 field, untuk Media) dan Mentah (4 field tambahan seperti source_ref_id, latitude, longitude, attributes, untuk scope lebih tinggi). Permintaan field yang ada di daftar Mentah tanpa scope yang sesuai mengembalikan 403 secara eksplisit. Permintaan field yang tidak dikenal sama sekali, di luar kedua daftar itu, akan mengembalikan 400.

7.2 Alasan dan Alternatif

Keputusan	Alternatif yang dipertimbangkan	Trade-off

Penandatanganan asimetris (EdDSA)	HMAC (kunci simetris tunggal)	HMAC lebih sederhana, tetapi kunci yang sama harus dibagi ke setiap verifier, bila satu service bocor, kunci untuk memalsukan token ikut bocor. EdDSA membuat client-facing-api hanya memegang kunci publik, tidak bisa memalsukan token meski diserang.
Access token pendek (60s) + refresh token (600s)	Satu token tunggal berumur panjang	Lebih banyak request refresh dan kompleksitas rotasi, tetapi jendela pakai token yang bocor jauh lebih kecil.
Sinkronisasi revocation lewat polling berkala	Verifikasi langsung ke auth-service tiap request	Opsi langsung lebih real-time, tetapi membuat auth-service jadi single point of failure dan menambah latensi tiap panggilan. Polling membuat client-facing-api tetap mandiri, dengan jendela singkat token baru dicabut masih diterima.
Fail-open saat sinkronisasi revocation gagal	Fail-closed (tolak semua token bila sinkronisasi gagal)	Fail-closed lebih aman secara teori, tetapi membuat seluruh sistem lumpuh hanya karena auth-service down sementara, padahal token yang sedang dipakai mungkin masih sah.
Reuse refresh token mencabut seluruh sesi	Hanya mencabut refresh token yang dipakai ulang	Opsi ringan lebih ramah pengguna, tetapi reuse adalah indikator kuat token dicuri, mencabut seluruh sesi lebih aman meski klien asli harus login ulang.

Tabel 17. Keputusan desain Problem 3 dan trade-off-nya

7.3 Batasan yang disadari

- Status sesi dan refresh token disimpan di memori proses auth-service, bukan database persistent. Restart auth-service menghapus seluruh sesi aktif, semua klien harus login ulang. Diterima karena skala M1 masih satu instans tanpa kebutuhan high availability.
- Sinkronisasi revocation ke client-facing-api bersifat polling, bukan push instan, ada jendela waktu singkat di mana token yang baru dicabut masih diterima sampai polling berikutnya.
- Percobaan login gagal dibatasi 10 kali per menit per alamat IP penelepon, setelah itu /token menjawab 429 dengan Retry-After: 60 selama jendela waktu berjalan, tanpa memeriksa kredensial. Hitungan disimpan di memori auth-service dan hilang saat restart. Pembatasan berbasis IP dapat memengaruhi banyak client yang berbagi alamat yang sama.
- Tidak ada mutual TLS antar service pada M1. Trust antar service murni berbasis token JWT dan internal key pada header, bukan sertifikat.

- auth-service berjalan sebagai instans tunggal, tidak ada replikasi, konsisten dengan batasan single-instance pada keseluruhan sistem M1.

7.4 Bukti Penyelesaian

Bukti 1: Validasi Kredensial per Domain Trust (BMKG, PVMBG)

Tujuan pengujian

Membuktikan bahwa bmkg-mock dan pvmbg-mock menolak permintaan yang tidak membawa kredensial domain yang benar, dan membedakan antara kredensial yang hilang (401) dengan kredensial yang ada tapi salah nilai (403).

Perintah pengujian

```
# BMKG tanpa header → 401
curl.exe -i $BMKG/seismic-events

# BMKG dengan kredensial PVMBG (header salah sama sekali) → 401
curl.exe -i -H "Authorization: Bearer pvmbg-demo-token" $BMKG/seismic-events

# BMKG dengan header benar tapi nilai salah → 403
curl.exe -i -H "X-BMKG-Key: nilai-yang-salah" $BMKG/seismic-events

# PVMBG tanpa header → 401
curl.exe -i $PVMBG/volcanic-reports

# PVMBG dengan kredensial BMKG (header salah sama sekali) → 401
curl.exe -i -H "X-BMKG-Key: bmkg-demo-key" $PVMBG/volcanic-reports

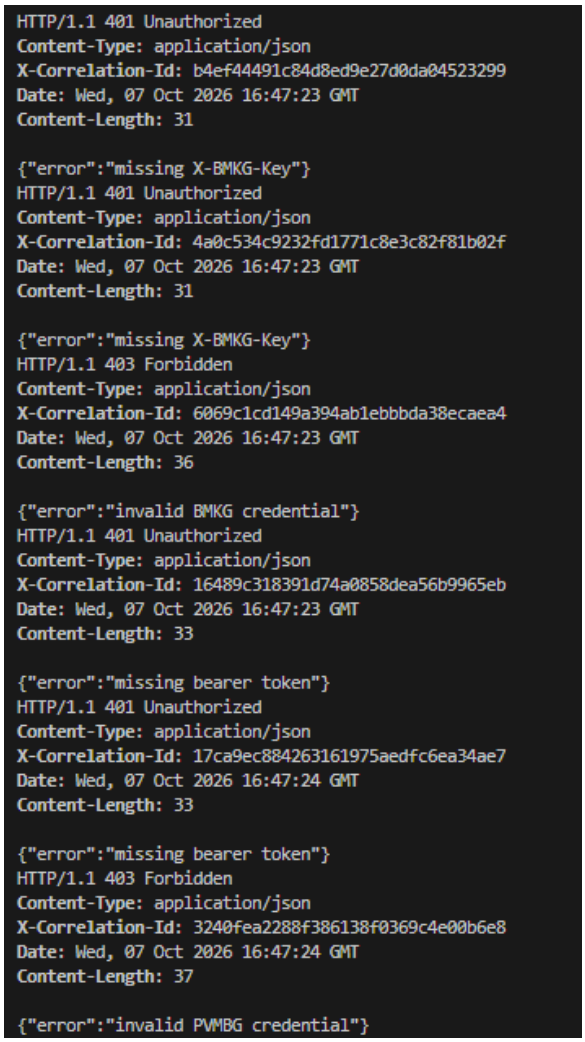
# PVMBG dengan header benar tapi nilai salah → 403
curl.exe -i -H "Authorization: Bearer token-salah" $PVMBG/volcanic-reports
```

Hasil

Skenario	Status	Pesan
BMKG tanpa header	401	missing X-BMKG-Key
BMKG pakai header Authorization (bukan X-BMKG-Key)	401	missing X-BMKG-Key
BMKG pakai X-BMKG-Key tapi nilai salah	403	invalid BMKG credential
PVMBG tanpa header	401	missing bearer token
PVMBG pakai X-BMKG-Key (bukan Authorization)	401	missing bearer token

PVMBG pakai Authorization tapi nilai salah	403	invalid PVMBG credential

Tabel 18. Hasil validasi kredensial lintas domain trust



Gambar 19. Respons validasi kredensial BMKG dan PVMBG

Kesimpulan

Setiap domain upstream (BMKG, PVMBG) hanya menerima skema header miliknya sendiri. Mengirim kredensial domain lain diperlakukan sama seperti tidak mengirim kredensial sama sekali (401), sementara kredensial dengan skema benar namun nilai salah baru menghasilkan 403. Ini menunjukkan keempat domain trust pada sistem (BMKG, PVMBG, Internal BNPB, Client) terisolasi satu sama lain dan tidak bisa saling menggantikan.

Bukti 2: Scope-Based Field Projection pada client-facing-api

Tujuan pengujian

Membuktikan bahwa akses ke field data dibatasi sesuai scope milik client, dan sistem membedakan antara field yang ada tapi di luar scope (403 forbidden_field) dengan field yang memang tidak dikenal (400 unknown_field).

Perintah pengujian

```
$mediaResp = curl.exe -s -X POST "$AUTH/token" -d
"grant_type=client_credentials&client_id=media-demo&client_secret=media-demo-s
ecret" | ConvertFrom-Json

$mediaToken = $mediaResp.access_token

curl.exe -i -H "Authorization: Bearer $mediaToken" "$API/hazard-events"
curl.exe -i -H "Authorization: Bearer $mediaToken"
"$API/hazard-events?fields=attributes"
curl.exe -i -H "Authorization: Bearer $mediaToken"
"$API/hazard-events?fields=foo"

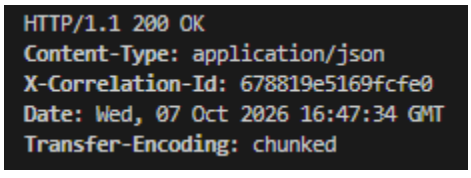
$fieldResp = curl.exe -s -X POST "$AUTH/token" -d
"grant_type=client_credentials&client_id=field-demo&client_secret=field-demo-s
ecret" | ConvertFrom-Json

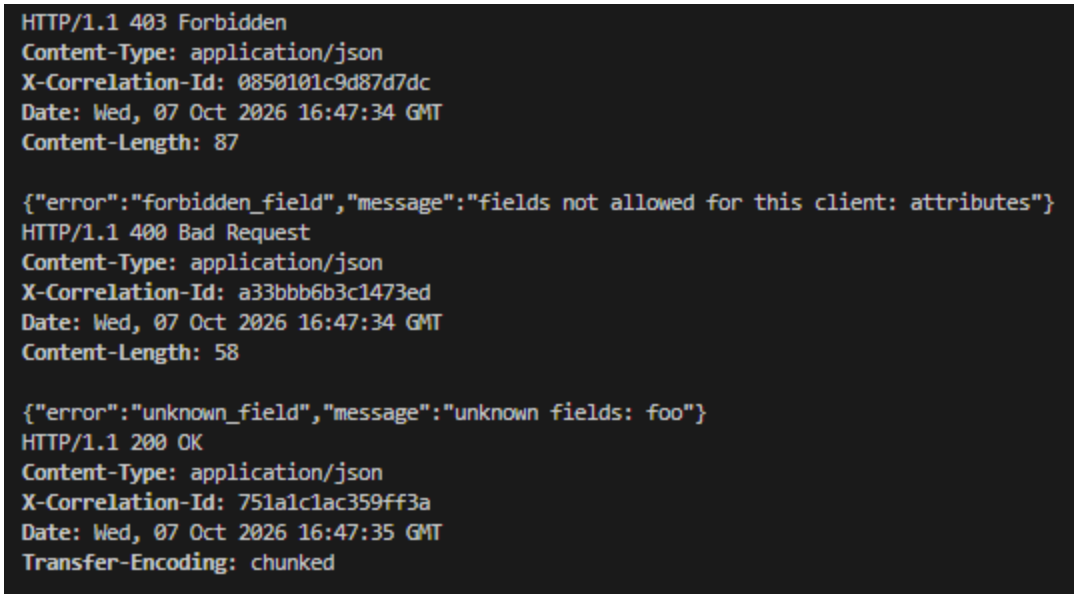
$fieldToken = $fieldResp.access_token
curl.exe -i -H "Authorization: Bearer $fieldToken" "$API/hazard-events"
```

Hasil

Client	Request	Status	Keterangan
media-demo (scope read:summary)	/hazard-events	200	hanya 7 field summary (tanpa attributes, latitude, longitude, source_ref_id)
media-demo	/hazard-events?fields=attributes	403	forbidden_field, "fields not allowed for this client: attributes"
media-demo	/hazard-events?fields=foo	400	unknown_field, "unknown fields: foo"
field-demo (scope read:summary read:raw)	/hazard-events	200	11 field lengkap, termasuk attributes, latitude, longitude, source_ref_id

Tabel 19. Hasil pengujian proyeksi field berdasarkan scope





Gambar 20. Respons proyeksi field berdasarkan scope client

Kesimpulan

Scope ditentukan sepenuhnya oleh server berdasarkan identitas client (bukan diberikan oleh client saat login), dan client-facing-api secara konsisten membedakan field yang ada tapi di luar izin (403) dengan field yang sama sekali tidak dikenal sistem (400).

Bukti 3: Lifecycle Token (Penerbitan, Expiry, Refresh)

Tujuan pengujian

Membuktikan bahwa access token kedaluwarsa sesuai TTL yang dikonfigurasi (AUTH_ACCESS_TTL_SECONDS=10 untuk keperluan demo, untuk bukti pengujian AUTH_ACCESS_TTL_SECONDS=60 terdapat pada Lampiran C), dan refresh token dapat dipakai untuk menerbitkan access token baru yang valid, sementara access token lama tetap tertolak.

Perintah pengujian

```
$t1 = curl.exe -s -X POST "$AUTH/token" -d
"grant_type=client_credentials&client_id=field-demo&client_secret=field-demo-s
ecret" | ConvertFrom-Json
$access1 = $t1.access_token
$refresh1 = $t1.refresh_token

curl.exe -i -H "Authorization: Bearer $access1" "$API/hazard-events"

Start-Sleep -Seconds 11

curl.exe -i -H "Authorization: Bearer $access1" "$API/hazard-events"
```

```
$t2 = curl.exe -s -X POST "$AUTH/token" -d
"grant_type=refresh_token&refresh_token=$refresh1" | ConvertFrom-Json
$access2 = $t2.access_token
$refresh2 = $t2.refresh_token

curl.exe -i -H "Authorization: Bearer $access2" "$API/hazard-events"
curl.exe -i -H "Authorization: Bearer $access1" "$API/hazard-events"
```

Hasil

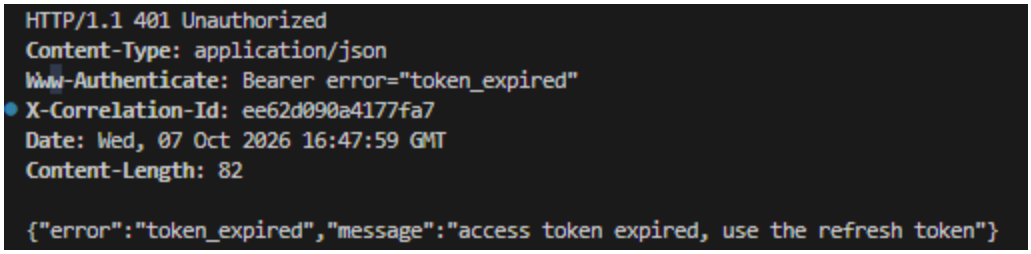
Langkah	Status	Keterangan
access1 dipakai segera setelah login	200	data hazard-events keluar normal
access1 dipakai setelah 11 detik (TTL 10 detik)	401	token_expired, header Www-Authenticate: Bearer error="token_expired"
Refresh pakai refresh1	200	server menerbitkan access2 dan refresh2 baru
access2 (token baru) dipakai	200	data hazard-events keluar normal
access1 (token lama) dicoba lagi	401	token_expired: ditolak karena sudah kedaluwarsa (bukan karena dicabut)

Tabel 20. Hasil pengujian lifecycle token

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: 3945adace7f2265c
Date: Wed, 07 Oct 2026 16:47:48 GMT
Transfer-Encoding: chunked
```

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Www-Authenticate: Bearer error="token_expired"
X-Correlation-Id: bfcbfeda862fe21f
Date: Wed, 07 Oct 2026 16:47:59 GMT
Content-Length: 82

{"error":"token_expired","message":"access token expired, use the refresh token"}
HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: 85948d131600a40f
Date: Wed, 07 Oct 2026 16:47:59 GMT
Transfer-Encoding: chunked
```



Gambar 21. Respons penerbitan, kedaluwarsa, dan refresh access token

Kesimpulan

Access token mematuhi TTL yang dikonfigurasi dan tidak bisa dipakai lagi setelah kedaluwarsa. Mekanisme refresh berhasil menerbitkan pasangan token baru yang valid tanpa mengharuskan client login ulang dengan client credentials.

Bukti 4: Deteksi Reuse Refresh Token

Tujuan pengujian

Membuktikan bahwa refresh token hanya bisa dipakai satu kali. Jika refresh token yang sama dipakai lagi (reuse) setelah pernah dipakai sebelumnya, seluruh sesi turunannya (termasuk access token yang masih berlaku dari sisi TTL) ikut dicabut, bukan hanya permintaan refresh itu sendiri yang ditolak.

Perintah pengujian

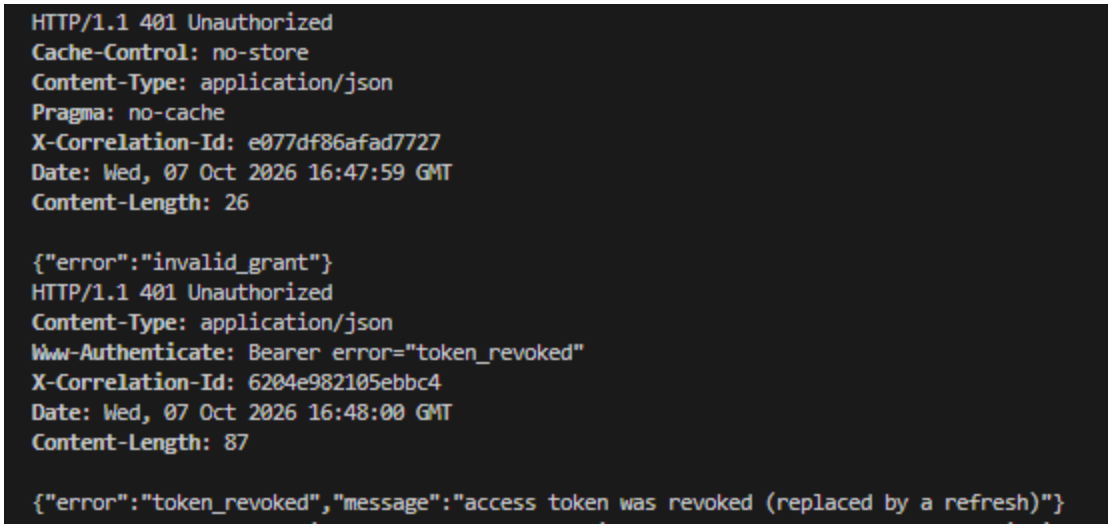
Lanjutan langsung dari Bukti 3, memakai refresh1 (yang sudah pernah dipakai) dan access2 (access token hasil refresh yang masih valid).

```
curl.exe -i -X POST "$AUTH/token" -d
"grant_type=refresh_token&refresh_token=$refresh1"
Start-Sleep -Milliseconds 1200
curl.exe -i -H "Authorization: Bearer $access2" "$API/hazard-events"
```

Hasil

Langkah	Status	Keterangan
refresh1 dipakai ulang (reuse)	401	invalid_grant, auth-service menolak refresh token yang sudah pernah dipakai
access2 dicek ulang (masih dalam ***TTL (Time to Live)***, jeda ~ 1,.2 detik)	401	token_revoked, header Www-Authenticate: Bearer error="token_revoked", pesan "access token was revoked (replaced by a refresh)"

Tabel 21. Hasil pengujian deteksi penggunaan ulang refresh token



Gambar 22. Hasil deteksi penggunaan ulang refresh token

Kesimpulan

Upaya reuse refresh token langsung ditolak di endpoint /token, dan seluruh access token turunan dari sesi tersebut ikut dicabut meskipun secara TTL belum kedaluwarsa. Jeda singkat diperlukan di antara permintaan reuse dan pengecekan ulang karena client-facing-api melakukan polling berkala ke GET /revocations (bukan menerima notifikasi langsung), sehingga ada delay propagasi singkat sebelum status revoked terlihat di sisi client-facing-api.

Bukti 5: Fail-Open Resilience saat auth-service Tidak Tersedia

Tujuan pengujian

Membuktikan bahwa client-facing-api tetap dapat memverifikasi token secara lokal (memakai public key dan revocation list yang sudah di-cache) ketika auth-service tidak dapat dihubungi, dan pulih secara otomatis begitu auth-service hidup kembali.

Perintah pengujian

```
$t3 = curl.exe -s -X POST "$AUTH/token" -d
"grant_type=client_credentials&client_id=field-demo&client_secret=field-demo-s
ecret" | ConvertFrom-Json
$access3 = $t3.access_token

docker compose stop auth-service

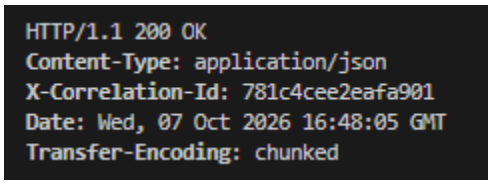
curl.exe -i -H "Authorization: Bearer $access3" "$API/hazard-events"
curl.exe -i -X POST "$AUTH/token" -d
"grant_type=client_credentials&client_id=field-demo&client_secret=field-demo-s
ecret"
docker compose logs client-facing-api --tail 50
```

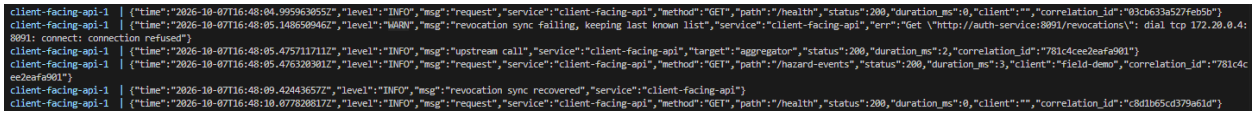
```
docker compose start auth-service
Start-Sleep -Seconds 5
docker compose logs client-facing-api --tail 20
```

Hasil

Langkah	Hasil
access3 dipakai setelah auth-service dimatikan	200, data tetap keluar normal
Login baru dicoba ke auth-service yang mati	curl: (52) Empty reply from server (wajar, auth-service sendiri yang tidak merespons)
Log client-facing-api saat auth-service mati	level=WARN msg="revocation sync failing, keeping last known list" err="dial tcp ...:8091: connect: connection refused"
Log client-facing-api setelah auth-service hidup kembali	level=INFO msg="revocation sync recovered"

Tabel 22. Hasil pengujian fail-open saat auth-service tidak tersedia





Gambar 23. Ketahanan client-facing-api dan pemulihan sinkronisasi revocation

Kesimpulan

client-facing-api tidak bergantung langsung pada ketersediaan auth-service untuk memverifikasi token yang sudah diterbitkan, karena verifikasi JWT dan pengecekan revocation list dilakukan secara lokal memakai salinan yang di-cache. Kegagalan sinkronisasi dicatat secara eksplisit (fail-open, bukan fail-closed) dan sistem pulih otomatis tanpa intervensi manual begitu konektivitas ke auth-service kembali normal.

Bukti 6: Credential Hygiene

Tujuan pengujian

Membuktikan bahwa file berisi secret asli (.env) tidak pernah ter*commit* ke repository, dan template yang di*commit* (.env.example) hanya berisi placeholder, bukan nilai rahasia sungguhan.

Perintah pengujian

```
git ls-files | Select-String "\.env"
Get-Content .env.example | Select-String "AUTH_SIGNING_SEED"
```

Hasil

```
.env.example

AUTH_SIGNING_SEED=CHANGE_ME_openssl_rand_base64_32
```

Kesimpulan

Hanya .env.example yang ter-track oleh git, sedangkan .env asli yang berisi AUTH_SIGNING_SEED sungguhan (digenerate lewat openssl rand base64 32) tidak pernah masuk ke version control. Nilai pada .env.example berupa placeholder yang mengharuskan setiap developer men-generate signing seed sendiri, sehingga tidak ada kunci penandatanganan produksi yang tersebar lewat repository.

Bukti 7: Token lama ditolak setelah refresh (masih dalam TTL (*Time to Live*))

Tujuan pengujian untuk membuktikan bahwa setelah refresh access token lama ditolak karena dicabut (bukan karena kadaluarsa), sementara token baru diterima.

Hasil

Perintah pengujian (TTL default 60 detik, tidak perlu diubah)

```
LOGIN=$(curl -s -X POST localhost:8091/token -d grant_type=client_credentials -d
client_id=field-demo -d client_secret=field-demo-secret)
OLD=$(echo $LOGIN | jq -r .access_token)
RT=$(echo $LOGIN | jq -r .refresh_token)
NEW=(curl − s − XPOSTlocalhost: 8091/token − dgrant_type = refresh_token − drefresh_token =RT | jq -r
.access_token)
sleep 2
curl -i -H "Authorization: Bearer $OLD" "localhost:8080/hazard-events?limit=1"
curl -i -H "Authorization: Bearer $NEW" "localhost:8080/hazard-events?limit=1"
```

Kesimpulan

Setelah refresh, permintaan dengan access token lama dijawab 401 token_revoked (header WWW-Authenticate: Bearer error="token_revoked") padahal jedanya hanya 2 detik, jauh di bawah TTL 60 detik, sedangkan permintaan dengan token baru dijawab 200. Jadi penolakan token lama disebabkan pencabutan, bukan kedaluwarsa. Jeda 2 detik diperlukan karena client-facing-api menyinkronkan daftar token yang dicabut secara berkala (maksimal sekitar 1 detik pada konfigurasi compose bawaan).

7.5 Jawaban Pertanyaan

Mekanisme yang dipakai

Empat domain trust terpisah dengan kredensial masing-masing (Bagian 7.1), JWT EdDSA berumur pendek dengan refresh token yang dirotasi dan dideteksi penggunaan ulangnya, scope yang ditetapkan di server, proyeksi field berbasis whitelist, dan throttle login per IP.

Mengapa mekanisme ini tidak membuka celah baru.

- Hanya algoritma EdDSA yang diterima, sehingga serangan downgrade (alg=none atau HMAC memakai kunci publik sebagai kunci simetris) gagal diverifikasi.
- client-facing-api hanya memegang kunci publik, kompromi pada client-facing-api tidak memungkinkan penerbitan token baru yang sah.
- Client secret dan refresh token tidak pernah disimpan dalam bentuk mentah, dan perbandingannya memakai constant-time comparison agar tidak bisa diukur lewat timing attack.
- Permintaan token dikirim form-encoded dan dibaca lewat PostForm, tidak pernah lewat query string yang berisiko tercatat di log akses, dan responsnya ditandai no-store agar tidak tersimpan di cache.
- Rotasi refresh token dengan deteksi reuse mencabut seluruh sesi bila token yang sudah dipakai muncul lagi, membatasi dampak pencurian token.
- TTL pendek (60 detik) membatasi jendela pakai token yang bocor, dan jendela revocation yang bergantung pada konfigurasi polling diakui secara eksplisit sebagai batasan (Bagian 7.3).
- Kebocoran satu domain trust tidak berlaku di domain lain, karena kredensialnya berbeda dan setiap pihak memvalidasi kredensialnya sendiri secara independen.

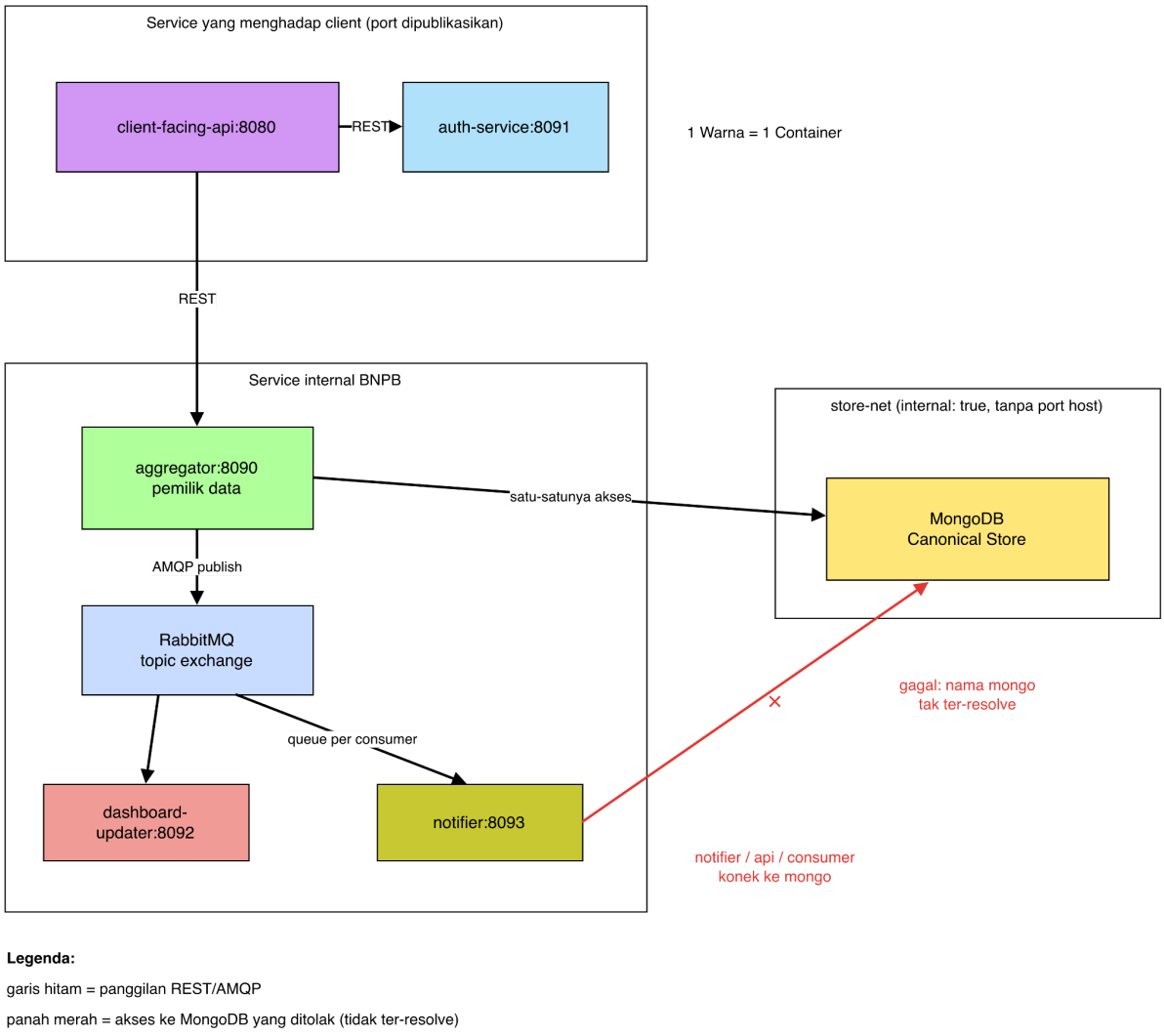
BAB 8

Problem 4: Arsitektur Microservice dan Flexible Storage

8.1 Rancangan

Dua masalah yang harus diatasi:

1. Sistem monolitik: mengubah satu bagian memaksa deploy semua.
2. Tabel relasional kaku: field baru berarti migrasi yang memaksa deploy bersama lagi. Solusinya adalah kumpulan service independen (Bab 2) dan penyimpanan dokumen fleksibel.



Gambar 24. Batas service dan kepemilikan Canonical Store

Independensi deploy

Setiap *service* memiliki Dockerfile dan go.mod sendiri. Membangun ulang satu *service* tanpa menyentuh yang lain:

```
docker compose up -d --build --no-deps auth-service
```

Opsi --no-deps mencegah Compose menyentuh *service* lain. Perilaku saat satu *service* mati:

- auth-service mati: *request* dengan token yang masih valid tetap dilayani (JWT diverifikasi lokal). Login dan *refresh* baru gagal, sinkronisasi *revocation* mencatat peringatan dan memakai daftar terakhir.

- aggregator mati: client-facing-api menjawab 503 eksplisit. *Consumer* tidak menerima *event* baru tetapi tetap hidup.
- notifier atau dashboard-updater mati: *ingest* dan *consumer* lain tidak terganggu (Bab 9).

Penyimpanan fleksibel: MongoDB

HazardEvent.attributes disimpan sebagai sub dokumen BSON bebas. Saat confidence_level muncul, dokumen baru langsung memiliki attributes.confidence_level sedangkan dokumen lama tidak, keduanya hidup berdampingan pada *collection* yang sama tanpa ALTER TABLE. Indeks dibuat otomatis oleh aggregator saat *start* (store/mongo.go) hanya pada kolom kanonik yang stabil: *unique* pada hazard_id (menjamin *upsert* idempoten), serta ingested_at, (hazard_type, ingested_at), dan (severity, ingested_at). Indeks ini bukan migrasi skema dan tidak berubah saat *field* baru muncul.

Canonical Store hanya dapat diakses aggregator

1. MongoDB tidak memiliki port ke host (tidak ada ports).
2. MongoDB hanya berada di jaringan store-net dengan internal: true, dan hanya aggregator yang ikut jaringan itu, *service* lain tidak dapat me *resolve* nama mongo.
3. MongoDB memakai autentikasi (MONGO_INITDB_ROOT_*) dan kredensialnya hanya dikirim ke aggregator.

Service lain yang membutuhkan data memintanya lewat GET /hazard-events aggregator (dilindungi X-Internal-Key).

8.2 Alasan dan Alternatif

Alasan penentuan batas service

Prinsip: memecah berdasarkan alasan berubah yang berbeda dan kepemilikan data.

- aggregator memiliki *ingest*, *mapping*, dan Canonical Store karena ketiganya berubah bersamaan (*mapping* bergantung bentuk data, penyimpanan bergantung hasil *mapping*). Memecahnya hanya menambah hop jaringan tanpa manfaat *deploy*.
- auth-service terpisah karena mengganti aturan *auth* tidak boleh memaksa *rebuild* logika *mapping*.
- client-facing-api terpisah dari aggregator karena menghadap pihak luar (permukaan serang) dan berprofil beban berbeda (*limiter*, *scope*), aggregator tidak perlu mengetahui client.
- *Consumer* (dashboard-updater, notifier) terpisah dan independen agar masing-masing dapat di *deploy* dan gagal sendiri sendiri.
- *Mock* terpisah karena merupakan entitas luar. Alternatif yang ditolak: satu service besar (monolit), atau memecah poller, mapper, dan store menjadi tiga service (terlalu halus: setiap perubahan skema menyentuh ketiganya, ditambah *latency* dan kompleksitas *deployment*).

Perbandingan alternatif penyimpanan

Opsi	Kelebihan	Kekurangan / konsekuensi
MongoDB (dipilih)	Atribut dinamis alami, *query* berdasar *field* kanonik tetap cepat dengan indeks, cocok untuk data berbentuk dokumen.	Konsistensi/transaksi lintas dokumen lebih lemah (tidak dibutuhkan: satu dokumen per *event*), lebih berat dari *key-value*, *query* ad-hoc di dalam attributes tanpa indeks lebih lambat.
PostgreSQL + JSONB	Kolom kanonik bertipe kuat plus JSONB untuk atribut, transaksi ACID, SQL matang.	Kolom kanonik tetap perlu migrasi bila berubah, perlu disiplin agar JSONB tidak menjadi tempat sampah, setup lebih banyak.
Key-value (mis. Redis)	Sangat cepat dan sederhana.	*Query filter* dan *sort* (hazard_type, severity, since) tidak alami, persistensi dan riwayat lebih lemah.

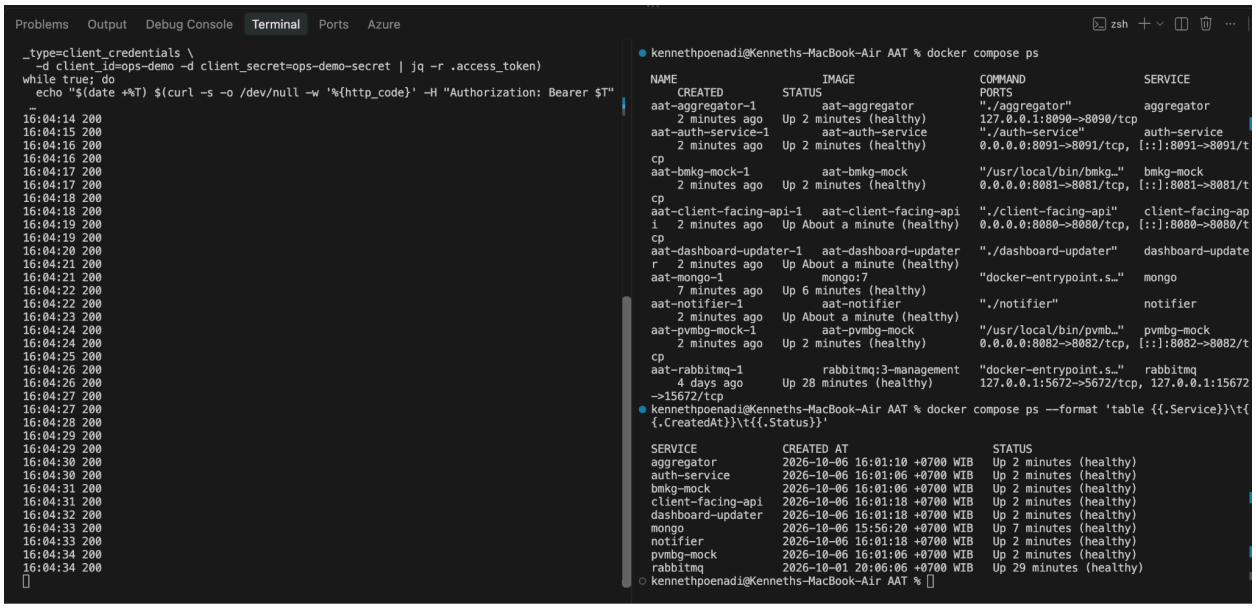
Tabel 23. Perbandingan alternatif penyimpanan

8.3 Batasan yang disadari

- Cursor polling di memori: setelah aggregator di-restart, cursor kosong sehingga seluruh riwayat diambil ulang dari mock. Upsert menjaga MongoDB tetap benar (tidak ada dokumen ganda), tetapi event lama dipublikasikan ulang ke broker, consumer menanganinya lewat dedupe (Bab 9).
- Instans tunggal: MongoDB, RabbitMQ, dan aggregator masing-masing berjalan sebagai satu *instance* pada satu host Docker Compose, tidak ada replikasi.
- Query ad-hoc pada isi attributes tidak diindeks (indeks hanya pada kolom kanonik).

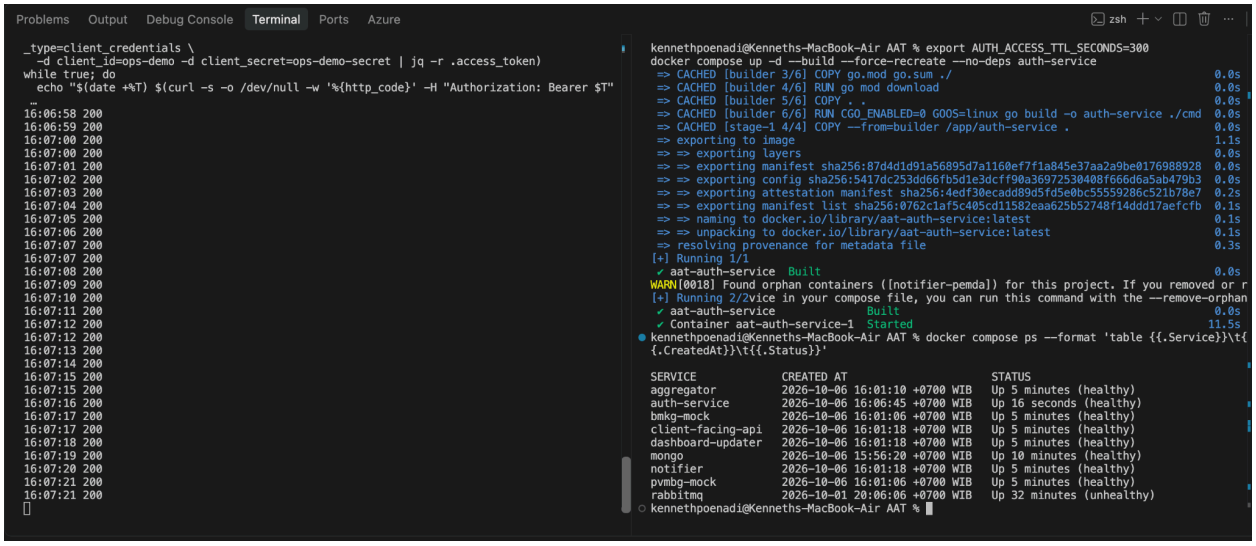
8.4 Bukti Penyelesaian

Bukti 1: Independensi deploy (rebuild satu service)



Gambar 25. Kondisi sebelum rebuild: loop curl (kiri) dan docker compose ps (kanan)

Terminal kiri menjalankan *loop* curl ke client-facing-api memakai satu *access token* dan mencetak kode HTTP setiap detik. Terminal kanan menampilkan docker compose ps: sembilan service berstatus healthy, semuanya dibuat pada 16:01 (mongo pada 15:56)



Gambar 26. Rebuild auth-service dengan --no-deps dan status service sesudahnya

auth-service dibangun ulang dan dijalankan ulang dengan docker compose up -d --build --force-recreate --no-deps auth-service. Hanya aat-auth-service yang berstatus *Built* dan *Started*. Pada docker compose ps berikutnya, hanya auth-service yang waktu pembuatannya berubah (16:06:45, uptime 16 detik), sedangkan *service* lain tetap 16:01 dengan *uptime* 5 menit sehingga tidak ikut di *restart*. TTL *access token* dinaikkan melalui AUTH_ACCESS_TTL_SECONDS=300 agar token pada loop tidak kadaluarsa selama pengujian.

Log kode HTTP dari loop (sekitar 16:04 sampai 16:08) seluruhnya 200, tanpa 401 atau 5xx, walaupun auth-service sedang dibangun ulang dan di *restart*. Ini membuktikan bahwa untuk mengubah satu service tidak memaksa deploy service lain, dan client-facing-api tetap melayani *token* yang masih *valid* karena JWT diverifikasi secara lokal tanpa memanggil auth-service.

Bukti hasil tetap 200 selama rebuild berlangsung

```
16:05:53 200
16:05:54 200
16:05:55 200
16:05:56 200
16:05:57 200
16:05:57 200
16:05:58 200
16:05:59 200
16:05:59 200
16:06:00 200
16:06:02 200
16:06:02 200
16:06:03 200
16:06:04 200
```

```
16:06:05 200
16:06:06 200
16:06:07 200
16:06:07 200
16:06:08 200
16:06:09 200
16:06:10 200
16:06:11 200
16:06:12 200
16:06:12 200
16:06:13 200
16:06:14 200
16:06:14 200
16:06:15 200
16:06:16 200
16:06:17 200
16:06:18 200
16:06:18 200
16:06:19 200
16:06:20 200
16:06:20 200
16:06:21 200
16:06:22 200
16:06:22 200
16:06:23 200
16:06:23 200
16:06:24 200
16:06:25 200
16:06:25 200
16:06:26 200
16:06:27 200
16:06:27 200
16:06:29 200
16:06:30 200
16:06:30 200
16:06:31 200
16:06:32 200
16:06:33 200
16:06:33 200
16:06:34 200
16:06:35 200
16:06:35 200
16:06:36 200
16:06:37 200
16:06:37 200
16:06:38 200
16:06:39 200
16:06:40 200
16:06:40 200
16:06:41 200
16:06:42 200
16:06:42 200
16:06:43 200
16:06:44 200
16:06:44 200
16:06:45 200
```

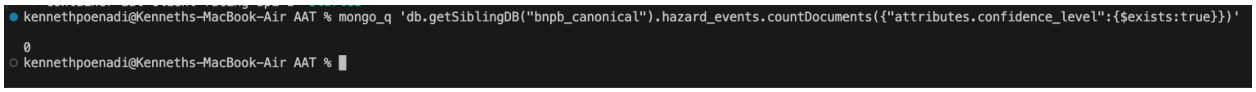
```
16:06:46 200
16:06:46 200
16:06:47 200
16:06:48 200
16:06:49 200
16:06:49 200
16:06:50 200
16:06:51 200
16:06:52 200
16:06:53 200
16:06:53 200
16:06:54 200
16:06:55 200
16:06:55 200
16:06:56 200
16:06:57 200
16:06:58 200
16:06:58 200
16:06:59 200
16:07:00 200
16:07:00 200
16:07:01 200
16:07:02 200
16:07:03 200
16:07:04 200
16:07:05 200
16:07:06 200
16:07:07 200
16:07:07 200
16:07:08 200
16:07:09 200
16:07:10 200
16:07:11 200
16:07:12 200
16:07:12 200
16:07:13 200
16:07:14 200
16:07:15 200
16:07:15 200
16:07:16 200
16:07:17 200
16:07:17 200
16:07:18 200
16:07:19 200
16:07:20 200
16:07:21 200
16:07:21 200
16:07:22 200
16:07:22 200
16:07:23 200
16:07:24 200
16:07:25 200
16:07:26 200
16:07:27 200
16:07:28 200
16:07:28 200
```

```
16:07:29 200
16:07:30 200
16:07:31 200
16:07:32 200
16:07:33 200
16:07:33 200
16:07:35 200
16:07:38 200
16:07:39 200
16:07:40 200
16:07:40 200
16:07:41 200
16:07:42 200
16:07:42 200
16:07:43 200
16:07:44 200
16:07:44 200
16:07:46 200
16:07:47 200
16:07:48 200
16:07:48 200
16:07:50 200
16:07:51 200
16:07:52 200
16:07:53 200
16:07:53 200
16:07:54 200
16:07:55 200
16:07:55 200
16:07:56 200
16:07:57 200
16:07:58 200
16:07:58 200
16:07:59 200
16:08:01 200
16:08:02 200
16:08:03 200
16:08:04 200
16:08:05 200
16:08:06 200
16:08:06 200
16:08:07 200
16:08:08 200
16:08:08 200
16:08:09 200
16:08:10 200
16:08:11 200
16:08:11 200
16:08:12 200
16:08:13 200
16:08:14 200
16:08:15 200
16:08:15 200
16:08:16 200
16:08:17 200
16:08:17 200
```

```
16:08:18 200
16:08:19 200
16:08:19 200
16:08:20 200
16:08:20 200
16:08:21 200
16:08:21 200
16:08:22 200
16:08:23 200
16:08:23 200
16:08:24 200
16:08:24 200
16:08:25 200
16:08:25 200
16:08:26 200
16:08:27 200
16:08:27 200
16:08:28 200
16:08:28 200
16:08:29 200
16:08:29 200
16:08:30 200
16:08:30 200
16:08:31 200
16:08:32 200
16:08:32 200
16:08:33 200
16:08:33 200
16:08:34 200
16:08:34 200
16:08:35 200
16:08:35 200
16:08:36 200
16:08:37 200
16:08:38 200
16:08:38 200
16:08:39 200
16:08:39 200
16:08:40 200
16:08:40 200
16:08:41 200
16:08:42 200
16:08:42 200
16:08:43 200
16:08:43 200
16:08:44 200
16:08:44 200
16:08:45 200
16:08:45 200
16:08:46 200
16:08:47 200
16:08:47 200
16:08:48 200
16:08:48 200
16:08:49 200
16:08:49 200
```

```
16:08:50 200
16:08:50 200
16:08:51 200
16:08:52 200
16:08:52 200
16:08:53 200
16:08:53 200
16:08:54 200
16:08:54 200
16:08:55 200
16:08:55 200
16:08:56 200
```

Bukti 2: Penyimpanan fleksibel (field baru tanpa migrasi)



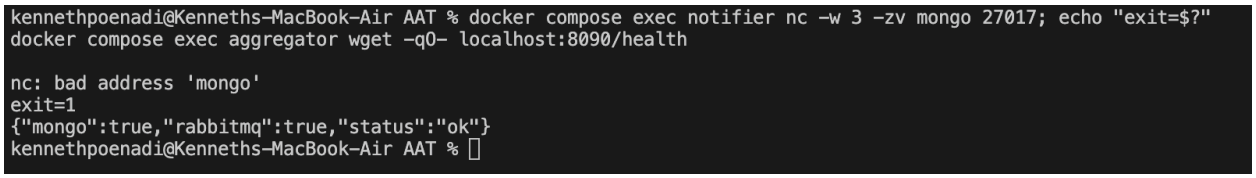
Gambar 27. Sebelum field baru muncul: jumlah dokumen dengan confidence_level = 0



Gambar 28. Setelah field baru muncul: dokumen lama dan baru berdampingan

Query pertama menghitung dokumen yang memiliki attributes.confidence_level dan menghasilkan 0, sebelum field baru muncul, koleksi belum memuatnya. Query kedua, setelah mock mulai mengirim confidence_level, menunjukkan dengan: 4 dan tanpa: 38. Dokumen baru (hz-pvmbg-000039) memiliki attributes.confidence_level: 0.93, sedangkan dokumen lama (hz-pvmbg-000038) tidak. Keduanya hidup berdampingan pada collection yang sama tanpa ALTER TABLE, migrasi, atau deploy ulang.

Bukti 3: Canonical Store hanya dapat diakses aggregator



Gambar 29. Notifier tidak dapat me-resolve mongo, aggregator dapat menjangkaunya

Perintah docker compose exec notifier nc -w 3 -zv mongo 27017 gagal dengan "nc: bad address 'mongo'" (exit=1). Notifier tidak berada pada jaringan store-net sehingga nama mongo tidak ter *resolve*. Sebaliknya, docker compose exec aggregator wget -qO- localhost:8090/health mengembalikan {"mongo":true,"rabbitmq":true,"status":"ok"}, yang berarti aggregator dapat menjangkau MongoDB.

8.5 Jawaban Pertanyaan

Alasan penentuan batas tiap *service*, alternatif pembagian yang ditolak, dan perbandingan tiga alternatif penyimpanan (MongoDB, PostgreSQL + JSONB, key-value) beserta konsekuensinya diuraikan pada Bagian 8.2. Ringkasnya: MongoDB dipilih karena atribut dinamis tertampung alami tanpa migrasi, dengan konsekuensi konsistensi lintas dokumen yang lebih lemah (tidak dibutuhkan karena satu dokumen per *event*) dan *query* ad-hoc pada *attributes* yang tidak terindeks.

BAB 9

Problem 5: *Fan-out Event Distribution/Pub-sub*

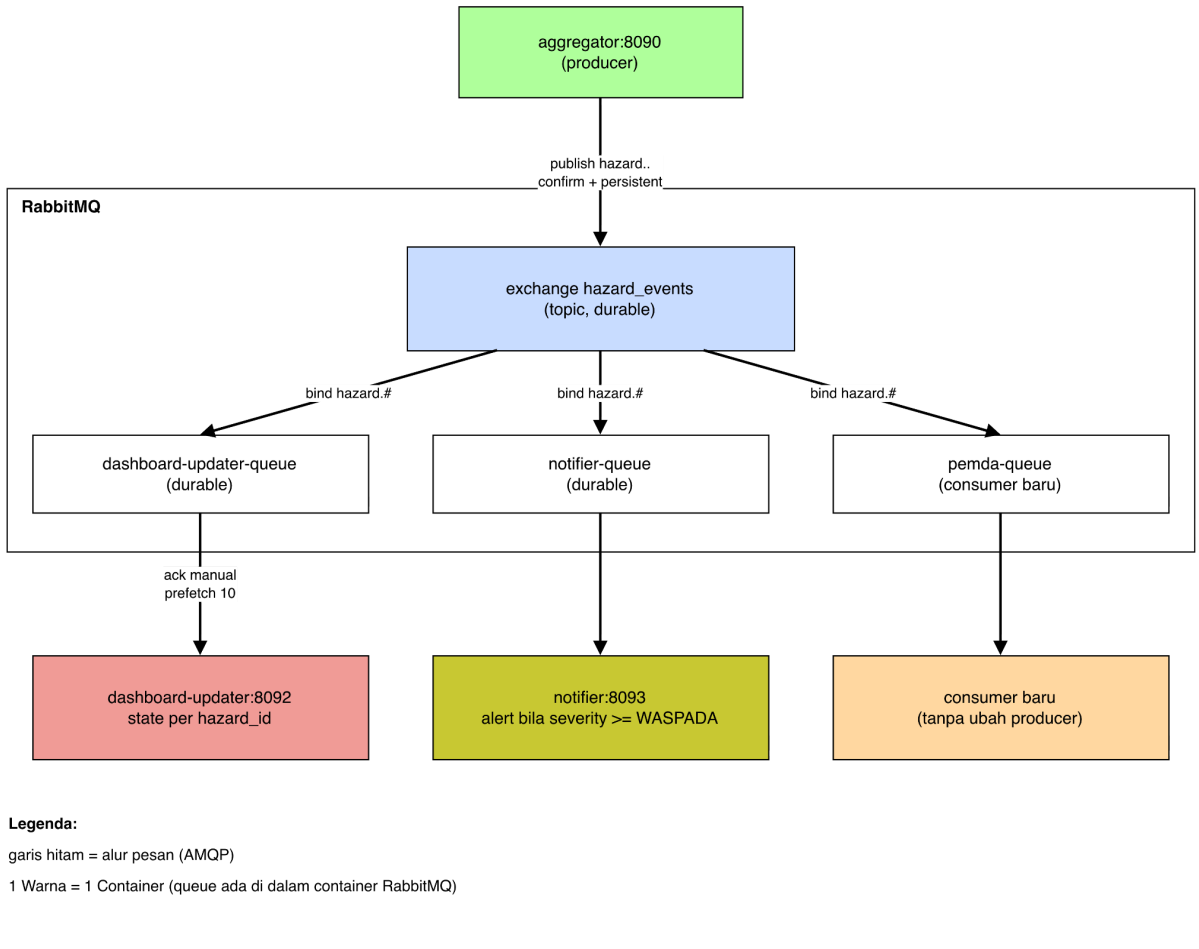
9.1 Rancangan

Masalah

Bila aggregator memanggil tiap pihak secara langsung, satu pihak lambat menahan yang lain dan setiap *consumer* baru berarti perubahan kode aggregator. Solusinya: letakkan *broker* di tengah.

Pola dan broker

RabbitMQ dengan *topic exchange* bernama hazard_events. *Producer* (aggregator) hanya mengirim ke *exchange* dengan *routing key* hazard.<type>.<severity> huruf kecil (contoh hazard.seismic.awas). Tiap *consumer* membuat *queue* sendiri (dashboard-updater-queue, notifier-queue) dan mengikatnya ke *exchange* dengan *binding key*, hazard.# berarti "semua", sedangkan *consumer* yang hanya peduli vulkanik dapat memakai hazard.volcanic.*. *Exchange* mengirim salinan tiap pesan ke setiap *queue* yang cocok (*fan-out*). *Producer* tidak mengetahui siapa yang berlangganan.



Gambar 30. Topologi pub-sub: producer, topic exchange, queue per consumer

Alur publish pada aggregator (broker/rabbitmq.go):

- Koneksi dibuat saat *start* dengan *retry* (RabbitMQ mungkin belum siap walau *container* sudah *healthy*).
- *Exchange* dideklarasikan *durable*, *channel* dalam mode publisher confirm: Publish menunggu konfirmasi *broker*, dan bila ditolak atau *timeout*, galat dikembalikan ke poller.
- Pesan bertanda *persistent* (DeliveryMode: Persistent) sehingga disimpan ke disk oleh *broker*.
- Bila koneksi putus, Publish menyambung ulang otomatis pada percobaan berikutnya, selama itu poller menahan *cursor* sehingga *event* tidak hilang.
- CorrelationId pesan diisi *correlation ID* siklus polling.

Alur pada consumer

1. Konek dan set Qos(prefetch=10): *broker* hanya mengirim maksimal 10 pesan yang belum di-*ack* (*backpressure*).
2. Deklarasi *queue durable* dan QueueBind ke *exchange*.
3. Consume dengan ack manual: pesan dianggap selesai setelah Ack, bila *consumer* mati sebelum *ack*, *broker* mengirim ulang (*redelivery*).
4. *Dedupe* dengan kunci hazard_id | severity pada SeenSet (FIFO 10.000 entri). Kunci menyertakan *severity* agar perubahan *severity* (mis. dikoreksi karena *warning* tsunami) tetap diproses.
5. Proses: dashboard-updater memperbarui *state* per hazard_id, notifier mencatat ALERT dispatched (simulated) bila *severity* ≥ NOTIFIER_MIN_SEVERITY, selain itu event received, below alert threshold.
6. Pesan tidak dapat di *parse* di Nack(requeue = false) agar tidak berputar selamanya.
7. Bila koneksi putus, *loop* di main menyambung ulang tiap 3 detik.

9.2 Alasan dan Alternatif

Broker	Penilaian
Kafka	Log terdistribusi dengan retensi, *consumer* baru dapat membaca ulang dari awal. Lebih berat dijalankan dan berlebihan untuk M1.
NATS *core*	Sangat ringan tetapi *at most once* (pesan hilang bila *consumer* mati). JetStream menambah *durability* dengan kompleksitas tambahan.
RabbitMQ (dipilih)	Routing fleksibel (*topic*), *queue durable* per *consumer*, *ack* manual, dan model mental mudah dijelaskan.

Tabel 24. Alternatif message broker

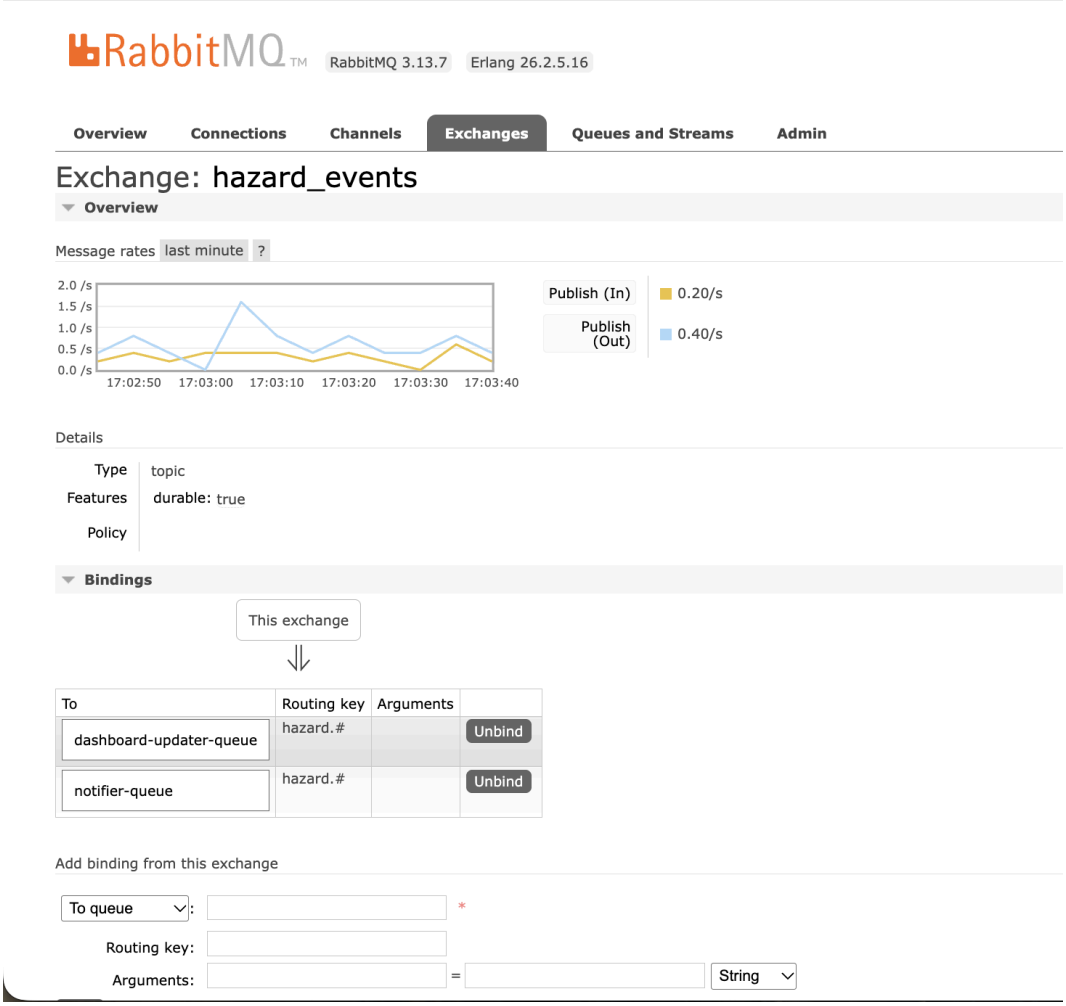
Alasan memilih *at least once* dibanding *at most once*: untuk peringatan bencana, kehilangan satu peringatan lebih buruk daripada menerima duplikat.

9.3 Batasan yang disadari

- Pada *cold start*, *data seed* yang diterbitkan aggregator pada *polling* pertama dapat tidak diterima consumer karena *queue consumer* baru dibuat setelah aggregator sehat (publish memakai mandatory = false, sehingga pesan tanpa *queue* tujuan dibuang broker). Pada pengujian, MongoDB berisi 125 dokumen sedangkan state dashboard-updater berisi 83 event (selisih 42, yaitu 40 data seed dan dua event awal). Data tetap utuh di Canonical Store dan terbaca lewat API. *Replay* ke *consumer* membutuhkan broker berbasis log seperti Kafka.
- Queue dibuat oleh consumer, bukan producer: *event* yang terbit sebelum *queue consumer* baru ada tidak ikut diterimanya. Replay dari awal membutuhkan *broker* berbasis log seperti Kafka.
- Dedupe consumer di memori: hilang saat *restart*, notifier dapat mengirim peringatan ganda setelah *restart*. Untuk dashboard-updater hal ini aman karena *state* per hazard_id idempoten.
- State dashboard-updater di memori: hanya berisi *event* yang diterima sejak *start* terakhir.
- Tanpa *dead letter queue*: pesan rusak dibuang (Nack tanpa *requeue*), tidak disimpan untuk diperiksa.
- RabbitMQ berjalan sebagai instans tunggal.

9.4 Bukti Penyelesaian

Bukti 1: Topologi pub-sub (topic exchange, binding, queue per consumer)



Gambar 31. Exchange hazard_events pada RabbitMQ Management

Halaman Exchanges menunjukkan hazard_events bertipe *topic* dengan durable: true. Pada bagian *Bindings*, dua *queue* terikat ke *exchange* dengan *routing* key hazard.#, yaitu dashboard-updater-queue dan notifier-queue. Setiap *consumer* memiliki *queue* sendiri, sehingga satu pesan yang cocok disalin ke kedua *queue* (fan out). Grafik *Message rates* memperlihatkan *Publish* (*In*) sekitar 0,20/s dan *Publish* (*Out*) sekitar 0,40/s, yaitu dua kali jumlah masuk karena tiap pesan keluar ke dua queue.

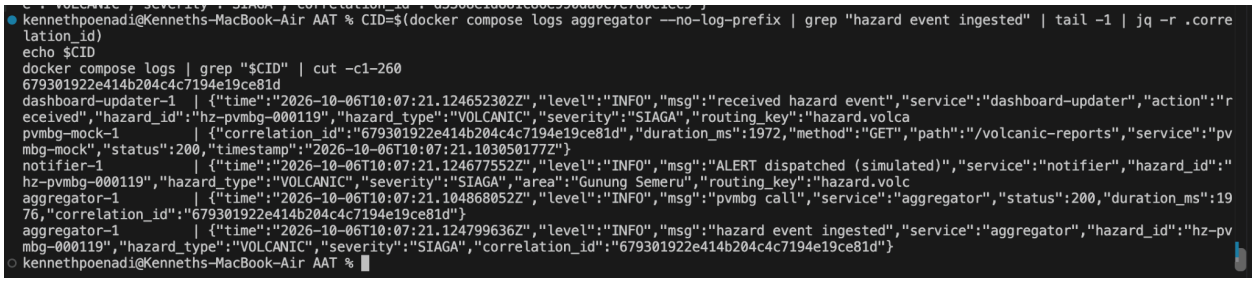
Bukti 2: Satu event sampai ke semua consumer (fan-out dan penelusuran correlation ID)

```
kennethpoenadi@Kenneths-MacBook-Air AAT % docker compose logs aggregator --no-log-prefix | grep "hazard event ingested" | tail -3

{"time":"2026-10-06T10:05:17.329723218Z","level":"INFO","msg":"hazard event ingested","service":"aggregator","hazard_id":"hz-pvmbg-000106","hazard_type":"VOLCANIC","severity":"WASPADA","correlation_id":"53a0837b2fa6d0e30994172476d70d5a"}
{"time":"2026-10-06T10:05:19.360190802Z","level":"INFO","msg":"hazard event ingested","service":"aggregator","hazard_id":"hz-bmkg-000107","hazard_type":"SEISMIC","severity":"WASPADA","correlation_id":"c000af2fda213f15de94bcaec894bdfe"}
{"time":"2026-10-06T10:05:22.141912345Z","level":"INFO","msg":"hazard event ingested","service":"aggregator","hazard_id":"hz-pvmbg-000107","hazard_type":"VOLCANIC","severity":"SIAGA","correlation_id":"d5368e1d881c86e990da0c7e7d0e1ec9"}
kennethpoenadi@Kenneths-MacBook-Air AAT %
```

Gambar 32. Log aggregator: event yang di-ingest beserta correlation_id

Log aggregator mencatat setiap event yang di-ingest beserta hazard_id, tipe, severity, dan correlation_id siklus polling.



Gambar 33. Satu correlation_id ditelusuri lintas service

Correlation ID dari *event* terakhir (679301922e414b204c4c7194e19ce81d) dicari pada *log* seluruh *service*. Hasilnya muncul berurutan pada pvmbg-mock (permintaan GET /volcanic-reports), aggregator (pvmbg *call* dan *hazard event ingested*), serta pada dashboard-updater (*received hazard event*) dan notifier (ALERT *dispatched*) untuk hazard_id yang sama, hz-pvmbg-000119 (VOLCANIC, SIAGA, *routing key* hazard.volcanic.siaga). Ini membuktikan bahwa satu event dipublikasikan sekali oleh aggregator dan diterima oleh kedua consumer, serta dapat ditelusuri dari sumber sampai *consumer*.

Bukti 3: dashboard-updater menyimpan state per hazard_id

```
kennethpoenadi@Kenneths-MacBook-Air AAT % docker compose exec
dashboard-updater wget -qO- localhost:8092/state | jq
{
  "total": 195,
  "by_severity": {
    "AWAS": 32,
    "NORMAL": 46,
    "SIAGA": 56,
    "WASPADA": 61
  },
  "by_type": {
    "SEISMIC": 97,
    "VOLCANIC": 98
  },
  "latest": [
    {
      "hazard_id": "hz-pvmbg-000131",
      "hazard_type": "VOLCANIC",
      "severity": "SIAGA",
      "area_name": "Gunung Semeru",
      "occurred_at": "2026-10-06T10:09:19.223532052Z",
      "received_at": "2026-10-06T10:09:20.827580261Z"
    },
    {
      "hazard_id": "hz-bmkg-000131",
      "hazard_type": "SEISMIC",
      "severity": "WASPADA",
      "area_name": "Pesisir Selatan Jawa",
      "occurred_at": "2026-10-06T10:09:18.932341051Z",
      "received_at": "2026-10-06T10:09:19.498343468Z"
    },
    {
      "hazard_id": "hz-pvmbg-000130",
```

```
      "hazard_type": "VOLCANIC",
      "severity": "WASPADA",
      "area_name": "Gunung Kelud",
      "occurred_at": "2026-10-06T10:09:09.223277588Z",
      "received_at": "2026-10-06T10:09:10.881544506Z"
    },
    {
      "hazard_id": "hz-bmkg-000130",
      "hazard_type": "SEISMIC",
      "severity": "WASPADA",
      "area_name": "Laut Banda",
      "occurred_at": "2026-10-06T10:09:08.93482588Z",
      "received_at": "2026-10-06T10:09:09.342786755Z"
    },
    {
      "hazard_id": "hz-pvmbg-000129",
      "hazard_type": "VOLCANIC",
      "severity": "NORMAL",
      "area_name": "Gunung Merapi",
      "occurred_at": "2026-10-06T10:08:59.22225775Z",
      "received_at": "2026-10-06T10:09:00.148883209Z"
    },
    {
      "hazard_id": "hz-bmkg-000129",
      "hazard_type": "SEISMIC",
      "severity": "WASPADA",
      "area_name": "Selat Sunda",
      "occurred_at": "2026-10-06T10:08:58.930586167Z",
      "received_at": "2026-10-06T10:08:59.348708625Z"
    },
    {
      "hazard_id": "hz-pvmbg-000128",
      "hazard_type": "VOLCANIC",
      "severity": "AWAS",
      "area_name": "Gunung Sinabung",
      "occurred_at": "2026-10-06T10:08:49.225639218Z",
      "received_at": "2026-10-06T10:08:50.798757969Z"
    },
    {
      "hazard_id": "hz-bmkg-000128",
      "hazard_type": "SEISMIC",
      "severity": "SIAGA",
      "area_name": "Sulawesi Utara",
      "occurred_at": "2026-10-06T10:08:48.935056676Z",
      "received_at": "2026-10-06T10:08:49.348940552Z"
    },
    {
      "hazard_id": "hz-pvmbg-000127",
      "hazard_type": "VOLCANIC",
      "severity": "SIAGA",
      "area_name": "Gunung Semeru",
      "occurred_at": "2026-10-06T10:08:39.225949005Z",
      "received_at": "2026-10-06T10:08:41.329917423Z"
    },
    {
      "hazard_id": "hz-bmkg-000127",
```

```
    "hazard_type": "SEISMIC",
    "severity": "WASPADA",
    "area_name": "Pesisir Selatan Jawa",
    "occurred_at": "2026-10-06T10:08:38.936580547Z",
    "received_at": "2026-10-06T10:08:39.353830089Z"
  },
  {
    "hazard_id": "hz-pvmbg-000126",
    "hazard_type": "VOLCANIC",
    "severity": "WASPADA",
    "area_name": "Gunung Kelud",
    "occurred_at": "2026-10-06T10:08:29.225427875Z",
    "received_at": "2026-10-06T10:08:31.275081085Z"
  },
  {
    "hazard_id": "hz-bmkg-000126",
    "hazard_type": "SEISMIC",
    "severity": "SIAGA",
    "area_name": "Laut Banda",
    "occurred_at": "2026-10-06T10:08:28.935713709Z",
    "received_at": "2026-10-06T10:08:29.35366525Z"
  },
  {
    "hazard_id": "hz-pvmbg-000125",
    "hazard_type": "VOLCANIC",
    "severity": "NORMAL",
    "area_name": "Gunung Merapi",
    "occurred_at": "2026-10-06T10:08:19.223631427Z",
    "received_at": "2026-10-06T10:08:20.651667094Z"
  },
  {
    "hazard_id": "hz-bmkg-000125",
    "hazard_type": "SEISMIC",
    "severity": "NORMAL",
    "area_name": "Selat Sunda",
    "occurred_at": "2026-10-06T10:08:18.934836635Z",
    "received_at": "2026-10-06T10:08:19.34281176Z"
  },
  {
    "hazard_id": "hz-pvmbg-000124",
    "hazard_type": "VOLCANIC",
    "severity": "AWAS",
    "area_name": "Gunung Sinabung",
    "occurred_at": "2026-10-06T10:08:09.22495238Z",
    "received_at": "2026-10-06T10:08:10.595178964Z"
  },
  {
    "hazard_id": "hz-bmkg-000124",
    "hazard_type": "SEISMIC",
    "severity": "WASPADA",
    "area_name": "Sulawesi Utara",
    "occurred_at": "2026-10-06T10:08:08.934528505Z",
    "received_at": "2026-10-06T10:08:09.361281964Z"
  },
  {
    "hazard_id": "hz-pvmbg-000123",
```

```
      "hazard_type": "VOLCANIC",
      "severity": "SIAGA",
      "area_name": "Gunung Semeru",
      "occurred_at": "2026-10-06T10:07:59.223916792Z",
      "received_at": "2026-10-06T10:08:01.937727127Z"
    },
    {
      "hazard_id": "hz-bmkg-000123",
      "hazard_type": "SEISMIC",
      "severity": "WASPADA",
      "area_name": "Pesisir Selatan Jawa",
      "occurred_at": "2026-10-06T10:07:58.93380925Z",
      "received_at": "2026-10-06T10:07:59.357556959Z"
    },
    {
      "hazard_id": "hz-pvmbg-000122",
      "hazard_type": "VOLCANIC",
      "severity": "WASPADA",
      "area_name": "Gunung Kelud",
      "occurred_at": "2026-10-06T10:07:49.222934427Z",
      "received_at": "2026-10-06T10:07:51.558870511Z"
    },
    {
      "hazard_id": "hz-bmkg-000122",
      "hazard_type": "SEISMIC",
      "severity": "SIAGA",
      "area_name": "Laut Banda",
      "occurred_at": "2026-10-06T10:07:48.948311468Z",
      "received_at": "2026-10-06T10:07:49.470780885Z"
    }
  ]
}
kennethpoenadi@Kenneths-MacBook-Air AAT %
```

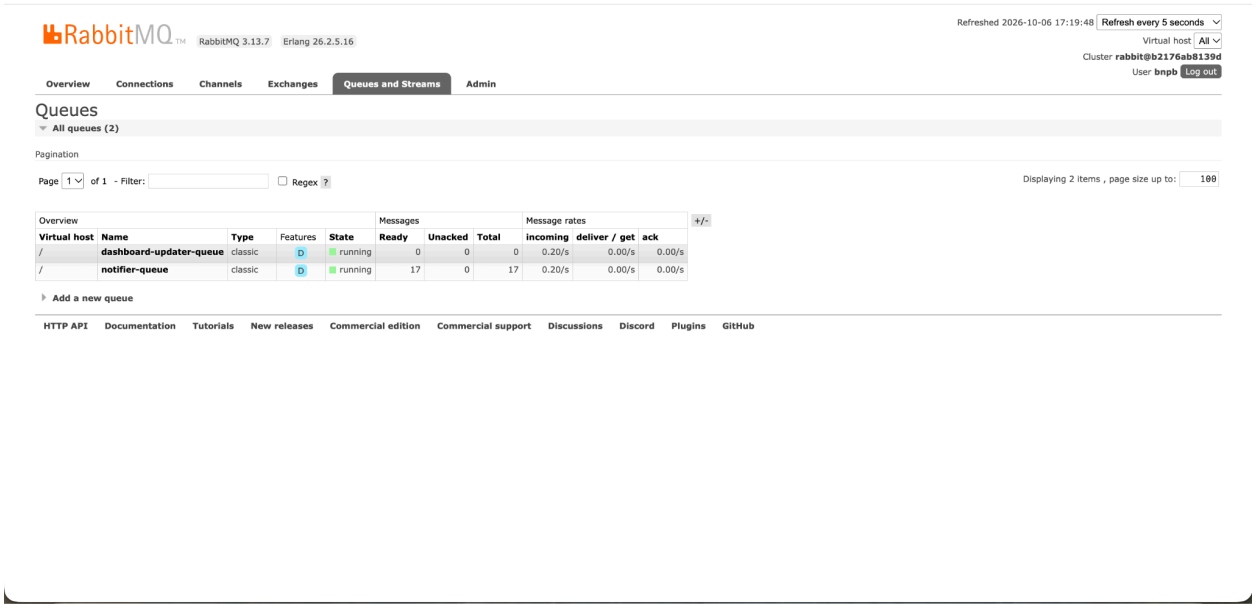

Endpoint /state pada dashboard-updater mengembalikan ringkasan dari seluruh *event* yang telah diterima: *total* 195 *event*, terbagi per *severity* dan per tipe, serta daftar *event* terbaru. Angka by_type berjumlah 195 (97 *seismic* dan 98 vulkanik), konsisten dengan total. *State* disimpan per hazard_id, sehingga *event* yang sama dikirim ulang tidak menambah total (*consumer idempotent*).

Bukti 4: Isolasi kegagalan consumer dan tidak ada pesan hilang



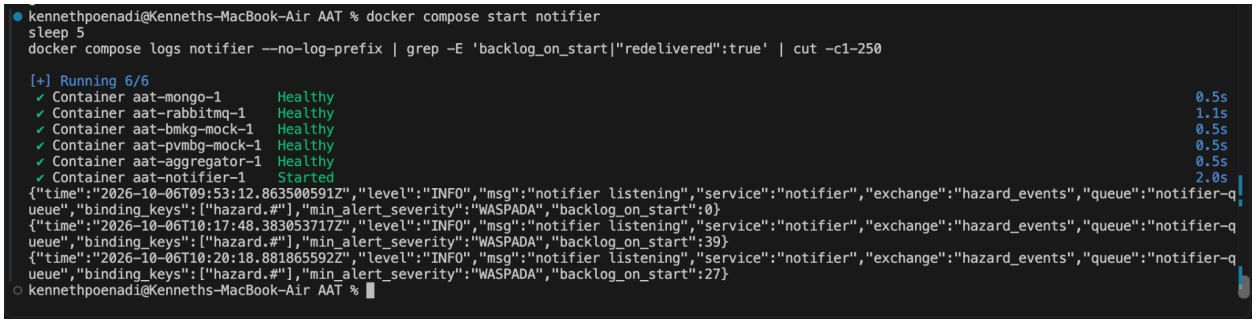
Gambar 34. Notifier dihentikan, Aggregator dan dashboard-updater tetap memproses event

Notifier dihentikan dengan docker compose stop notifier (Container aat-notifier-1 Stopped). Dalam 40 detik berikutnya aggregator tetap meng *ingest event* (12 entri *log*) dan dashboard-updater tetap menerima *event* (8 entri *log*). Gagalnya satu *consumer* tidak menghentikan *ingest* maupun *consumer* lain.



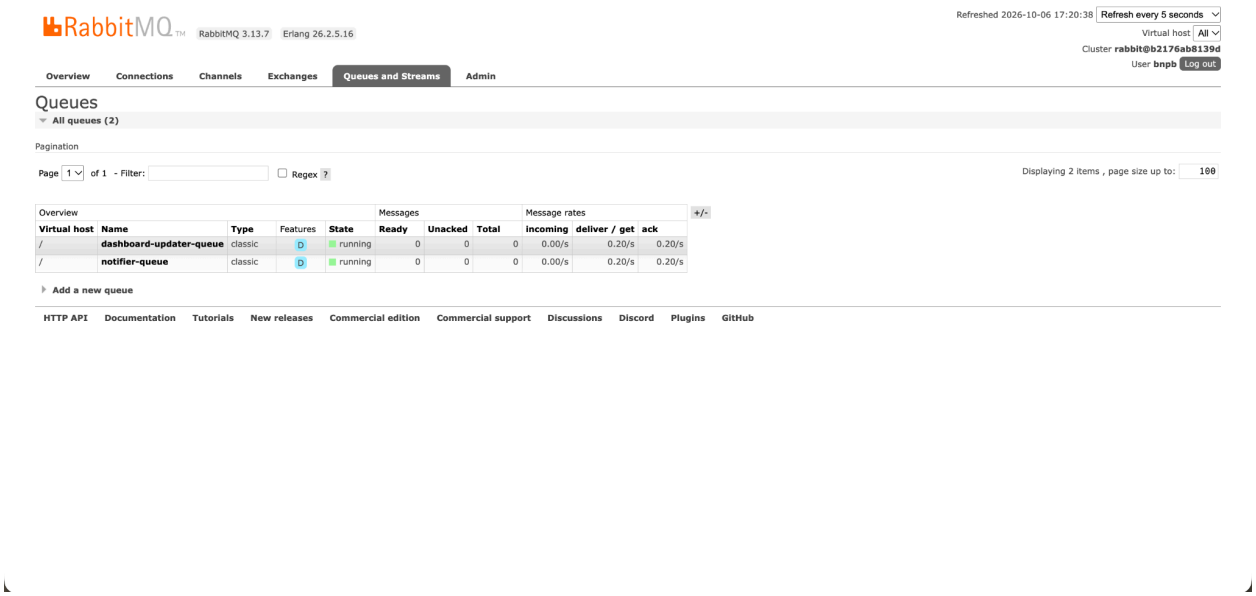
Gambar 35. RabbitMQ saat notifier mati: pesan tertahan di notifier-queue

Pada halaman Queues, notifier-queue menampung 17 pesan berstatus *Ready* (Total 17) karena tidak ada *consumer* yang mengambilnya, sedangkan dashboard-updater-queue tetap 0. Pesan tidak hilang: broker menyimpannya pada *queue durable* sampai notifier kembali.



Gambar 36. Notifier dinyalakan kembali: backlog_on_start

Setelah docker compose start notifier, *log notifier listening* mencatat backlog_on_start yang menunjukkan jumlah pesan yang menunggu saat *consumer* menyala (27 pada percobaan ini, nilai 0 dan 39 pada baris sebelumnya berasal dari *start* sebelumnya). Pesan-pesan tersebut diproses setelah *notifier* hidup kembali.

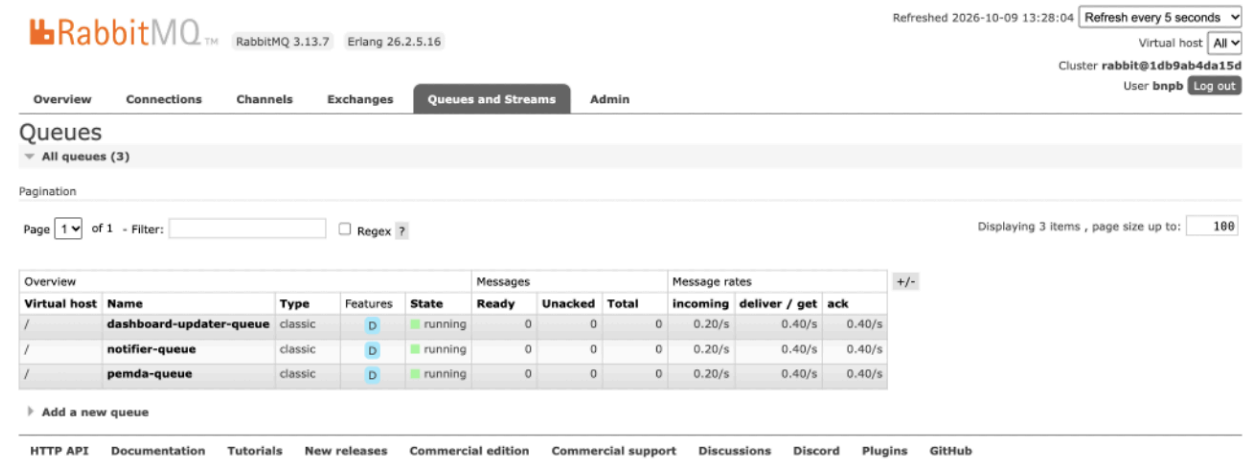


Gambar 37. Setelah notifier kembali: seluruh queue kosong dan acknowledgment berjalan

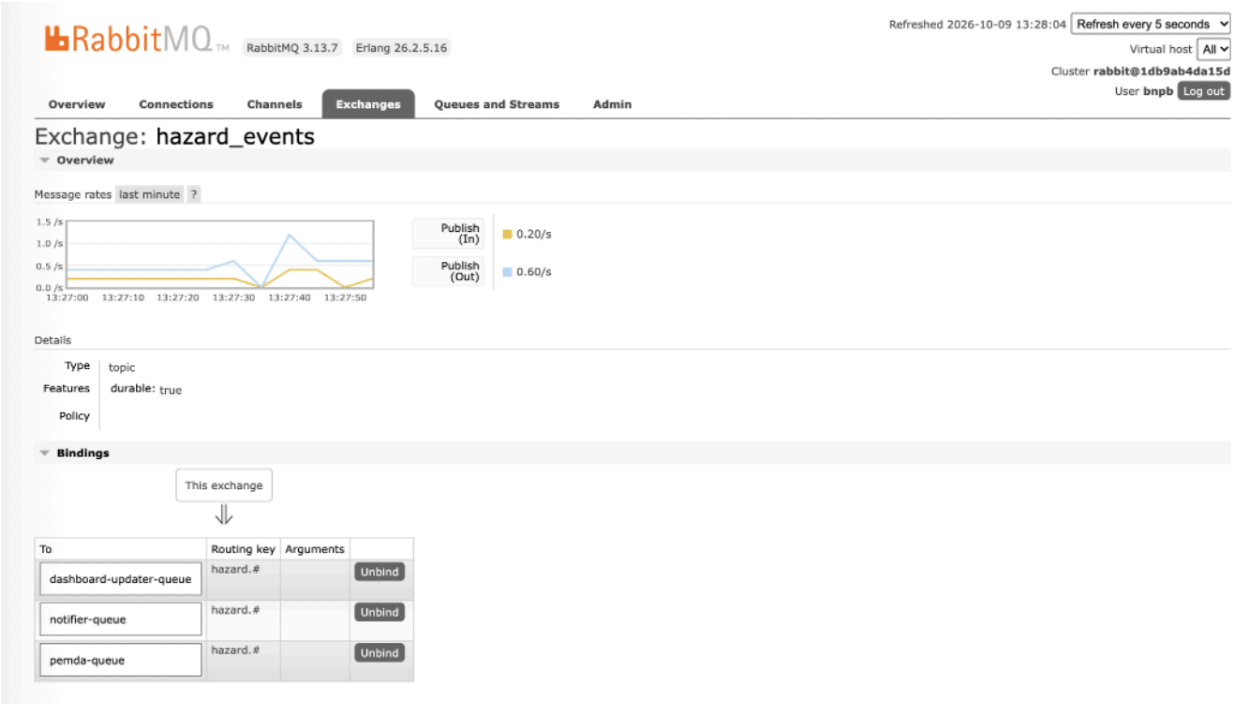
Setelah *notifier* hidup, halaman Queues menunjukkan kedua *queue* dengan *Ready* dan *Unacked* 0, serta *deliver/get* dan ack sama-sama 0,20/s. *Backlog* telah habis diproses dan setiap pesan yang dikirim di-ack, sehingga tidak ada pesan yang hilang selama notifier mati (semantik at least once).

Bukti 5: Penambahan consumer baru tanpa mengubah producer

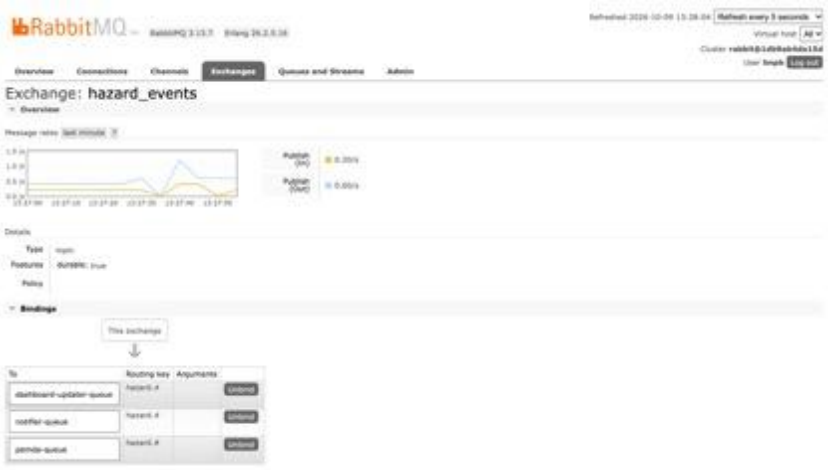
Consumer ketiga, notifier-pemda, dijalankan dari image notifier yang sama dengan nama queue berbeda (NOTIFIER_QUEUE=pemda-queue) memakai docker compose run -d --no-deps, tanpa menyentuh service lain. Pengujian dilakukan dua kali dengan prosedur identik: sekali untuk keluaran terminal (Gambar 40 dan 41) dan sekali untuk tampilan RabbitMQ Management (Gambar 38 dan 39).



Gambar 38a. Halaman Queues sebelum consumer baru: dua queue, dashboard-updater-queue dan notifier-queue.



Gambar 38b. Halaman Queues sesudah consumer baru: tiga queue, pemda-queue muncul.



Gambar 39. Exchange hazard_events (topic, durable) dengan tiga binding hazard.#, dan grafik Publish (In) 0,20/s berbanding Publish (Out) 0,60/s.

```
===== 1. SEBELUM: queue dan binding pada exchange hazard_events =====
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name     messages consumers
dashboard-updater-queue   0        1
notifier-queue   0        1

hazard_events     dashboard-updater-queue   hazard.#
hazard_events     notifier-queue   hazard.#

===== 2. SEBELUM: identitas container dan hash kode sumber aggregator =====
id=ee872119419425cc4bf7d6d34eb2dee7edbc9a2388f3f3ab3c9c6ef245dce000          dibuat=2026-10-09T06:10:28.89432626Z
mulai=2026-10-09T06:10:41.784067877Z restart_count=0
sha256 kode sumber aggregator: da2d6096afc61b585d440808fb57ff8c9acb780d2c63cbc1dc3671933582a937
```

Gambar 40a. Keluaran terminal sebelum consumer baru: queue, binding, identitas container aggregator, dan hash kode sumbernya.

```
===== 3. Menjalankan consumer baru (notifier-pemda) dengan queue baru (pemda-queue), tanpa menyentuh service
lain =====
$ docker compose run -d --no-deps --name notifier-pemda -e NOTIFIER_QUEUE=pemda-queue notifier
1abc6cc4e608beb96296b6f4f4c74559ee589682cb528d74eecf51b2c0f58839
menunggu 30 detik agar aggregator menerbitkan event baru...

===== 4. SESUDAH: queue dan binding (queue baru harus muncul dan terikat ke hazard_events) =====
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name     messages consumers
dashboard-updater-queue   0        1
pemda-queue       0        1
notifier-queue   0        1

hazard_events     dashboard-updater-queue   hazard.#
hazard_events     notifier-queue   hazard.#
hazard_events     pemda-queue       hazard.#

===== 5. Consumer baru menerima event dari stream yang sama =====
{"time":"2026-10-09T06:13:28.750161719Z","level":"INFO","msg":"ALERT                                    dispatched
(simulated)","service":"notifier","hazard_id":"hz-bmkg-000037","hazard_type":"SEISMIC","severity":"SIAGA","area"
:"Selat
Sunda","routing_key":"hazard.seismic.siaga","redelivered":false,"age_ms":175,"correlation_id":"29257ea80f01c9cb3
6844f0cfbea398e"}
{"time":"2026-10-09T06:13:28.954720635Z","level":"INFO","msg":"event        received,        below        alert
threshold","service":"notifier","hazard_id":"hz-pvmbg-000037","hazard_type":"VOLCANIC","severity":"NORMAL","area
":"Gunung
Merapi","routing_key":"hazard.volcanic.normal","redelivered":false,"age_ms":11,"correlation_id":"42182275daa77f4
5e2462ff2c893bf56"}
{"time":"2026-10-09T06:13:38.493907792Z","level":"INFO","msg":"ALERT                                    dispatched
(simulated)","service":"notifier","hazard_id":"hz-bmkg-000038","hazard_type":"SEISMIC","severity":"SIAGA","area"
:"Laut
Banda","routing_key":"hazard.seismic.siaga","redelivered":false,"age_ms":32,"correlation_id":"4c4a0cc6041bd75ff5
c9d45e24756318"}
{"time":"2026-10-09T06:13:39.936948001Z","level":"INFO","msg":"ALERT                                    dispatched
(simulated)","service":"notifier","hazard_id":"hz-pvmbg-000038","hazard_type":"VOLCANIC","severity":"WASPADA","a
rea":"Gunung
Kelud","routing_key":"hazard.volcanic.waspada","redelivered":false,"age_ms":22,"correlation_id":"f44651ef36146cd
d8de88cb1c009a13b"}
{"time":"2026-10-09T06:13:48.484239547Z","level":"INFO","msg":"event        received,        below        alert
threshold","service":"notifier","hazard_id":"hz-bmkg-000039","hazard_type":"SEISMIC","severity":"NORMAL","area":
"Pesisir                                                                                    Selatan
Jawa","routing_key":"hazard.seismic.normal","redelivered":false,"age_ms":26,"correlation_id":"f1f560b7b2564c0f83
92fc3c55a46673"}
{"time":"2026-10-09T06:13:49.918837173Z","level":"INFO","msg":"ALERT                                    dispatched
(simulated)","service":"notifier","hazard_id":"hz-pvmbg-000039","hazard_type":"VOLCANIC","severity":"SIAGA","are
a":"Gunung
Semeru","routing_key":"hazard.volcanic.siaga","redelivered":false,"age_ms":41,"correlation_id":"f137b4c19a26c422
a9dcf539ec453903"}
```

Gambar 40b. Keluaran terminal saat consumer baru dijalankan: queue dan binding sesudahnya, dan log notifier-pemda menerima event.

```
===== 6. Satu correlation_id yang sama muncul di aggregator, ketiga consumer =====
correlation_id: f137b4c19a26c422a9dcf539ec453903
--- aggregator
{"time":"2026-10-09T06:13:49.927526381Z","level":"INFO","msg":"rabbitmq
call","service":"aggregator","target":"rabbitmq","operation":"publish","duration_ms":24,"success":true,"hazard_i
d":"hz-pvmbg-000039","correlation_id":"f137b4c19a26c422a9dcf539ec453903"}
{"time":"2026-10-09T06:13:49.927588798Z","level":"INFO","msg":"hazard                                    event
ingested","service":"aggregator","hazard_id":"hz-pvmbg-000039","hazard_type":"VOLCANIC","severity":"SIAGA","corr
elation_id":"f137b4c19a26c422a9dcf539ec453903"}
--- notifier
{"time":"2026-10-09T06:13:49.92199859Z","level":"INFO","msg":"ALERT                                    dispatched
(simulated)","service":"notifier","hazard_id":"hz-pvmbg-000039","hazard_type":"VOLCANIC","severity":"SIAGA","are
a":"Gunung
Semeru","routing_key":"hazard.volcanic.siaga","redelivered":false,"age_ms":44,"correlation_id":"f137b4c19a26c422
a9dcf539ec453903"}
--- dashboard-updater
{"time":"2026-10-09T06:13:49.923184465Z","level":"INFO","msg":"received                                  hazard
event","service":"dashboard-updater","action":"received","hazard_id":"hz-pvmbg-000039","hazard_type":"VOLCANIC",
"severity":"SIAGA","routing_key":"hazard.volcanic.siaga","redelivered":false,"correlation_id":"f137b4c19a26c422a
9dcf539ec453903"}
--- notifier-pemda
{"time":"2026-10-09T06:13:49.918837173Z","level":"INFO","msg":"ALERT                                    dispatched
(simulated)","service":"notifier","hazard_id":"hz-pvmbg-000039","hazard_type":"VOLCANIC","severity":"SIAGA","are
a":"Gunung
Semeru","routing_key":"hazard.volcanic.siaga","redelivered":false,"age_ms":41,"correlation_id":"f137b4c19a26c422
a9dcf539ec453903"}

===== 7. SESUDAH: aggregator tidak berubah sama sekali =====
id=ee872119419425cc4bf7d6d34eb2dee7edbc9a2388f3f3ab3c9c6ef245dce000         dibuat=2026-10-09T06:10:28.89432626Z
mulai=2026-10-09T06:10:41.784067877Z restart_count=0
sha256 kode sumber aggregator: da2d6096afc61b585d440808fb57ff8c9acb780d2c63cbc1dc3671933582a937
HASIL: kode aggregator TIDAK berubah, container aggregator TIDAK di-restart

SERVICE             STATUS                    CREATED AT
aggregator        Up 3 minutes (healthy)     2026-10-09 13:10:28 +0700 WIB
auth-service      Up 3 minutes (healthy)     2026-10-09 13:10:25 +0700 WIB
bmkg-mock         Up 3 minutes (healthy)     2026-10-09 13:10:25 +0700 WIB
client-facing-api   Up 3 minutes (healthy)   2026-10-09 13:10:31 +0700 WIB
dashboard-updater   Up 3 minutes (healthy)   2026-10-09 13:10:31 +0700 WIB
mongo             Up 14 minutes (healthy)    2026-10-09 12:59:14 +0700 WIB
notifier          Up 3 minutes (healthy)     2026-10-09 13:10:31 +0700 WIB
pvmbg-mock        Up 3 minutes (healthy)     2026-10-09 13:10:25 +0700 WIB
rabbitmq          Up 3 minutes (healthy)     2026-10-09 13:10:25 +0700 WIB
```

Gambar 41. Satu correlation_id yang sama pada aggregator dan ketiga consumer, serta aggregator yang tidak berubah.

Sebelum consumer baru dijalankan, exchange hazard_events memiliki dua binding hazard.# (Gambar 38a, 40a). Setelah dijalankan, muncul pemda-queue dengan binding hazard.# pada exchange yang sama, sehingga jumlah queue dan binding menjadi tiga (Gambar 38b, 39, 40b). Consumer baru menerima event yang terbit setelah queue dibuat, misalnya hz-bmkg-000037 sampai hz-pvmbg-000039 (Gambar 40b). Event hz-pvmbg-000039 (VOLCANIC, SIAGA, routing key hazard.volcanic.siaga) dengan correlation_id f137b4c19a26c422a9dcf539ec453903 muncul pada log aggregator (rabbitmq call publish dan hazard event ingested), notifier, dashboard-updater, dan notifier-pemda (Gambar 41). Itu menunjukkan satu event dipublikasikan sekali dan disalin ke ketiga queue. Grafik exchange memperlihatkan Publish (Out) tiga kali Publish (In), sesuai tiga queue penerima (Gambar 39).

Aggregator tidak diubah: ID container, waktu mulai (2026-10-09T06:10:41.784067877Z), dan restart_count (0) sama sebelum dan sesudah, dan hash SHA-256 kode sumber non-test aggregator sama

(da2d6096afc61b585d440808fb57ff8c9acb780d2c63cbc1dc3671933582a937). Setelah pengujian, container dan queue uji dihapus sehingga queue kembali dua.

9.5 Jawaban Pertanyaan

Broker dan pola

Broker: RabbitMQ. Pola: *publish-subscribe* dengan *topic exchange* dan satu *queue durable* per *consumer*

Semantik pengiriman: at least once

Setiap *event* dijamin sampai sekali atau lebih, tidak pernah nol (selama sistem normal), tetapi dapat duplikat. Sumber duplika yaitu *cursor* ditahan lalu *batch* dipublikasikan ulang setelah kegagalan, *restart* aggregator (*cursor* kosong), *redelivery* RabbitMQ karena *consumer* mati sebelum *ack*, dan *event* dengan *severity* yang dikoreksi.

Konsekuensi bagi consumer: wajib idempoten

- dashboard-updater: *state* disimpan per hazard_id (*map*), sehingga menerapkan *event* yang sama dua kali tidak mengubah hasil.
- Keduanya memakai *dedupe* hazard_id|severity di memori. Keterbatasannya: *dedupe* hilang saat *restart*, sehingga *event* lama dapat diproses lagi. Untuk dashboard hal ini aman (idempoten), untuk notifier berarti kemungkinan peringatan ganda setelah *restart*. Hal ini dinyatakan sebagai batasan (9.3).

Menambah consumer baru tanpa mengubah producer

Consumer baru cukup membuat *queue* baru dan mengikatnya ke *exchange* yang sama. Pada pengujian, kode notifier dijalankan dengan nama queue berbeda (pemda-queue) sebagai consumer ketiga, seperti ditunjukkan pada Bukti 5 (Bagian 9.4). Hash SHA-256 kode sumber aggregator sama sebelum dan sesudah penambahan (Gambar 41).

```
docker compose run -d --no-deps --name notifier-pemda \ -e
NOTIFIER_QUEUE=pemda-queue notifier
```

Tidak ada satu baris kode aggregator pun yang berubah.

BAB 10

Petunjuk Menjalankan

10.1 Langkah Menjalankan

Ringkasan berikut merujuk ke README.md pada *repository*, yang memuat langkah menjalankan dari kondisi awal, daftar variabel konfigurasi, dan cara memicu setiap skenario pengujian.

```
# dari kondisi bersih
cp .env.example .env     # isi AUTH_SIGNING_SEED: openssl rand -base64 32
docker compose down -v && docker compose up -d --build
docker compose ps
```

Seluruh sistem dinyalakan dengan satu perintah orkestrasi (docker compose up -d --build). Urutan nyala diatur oleh healthcheck dan depends_on: condition: service_healthy. Variabel wajib yang kosong membuat Compose menolak berjalan.

10.2 Daftar Port

Service	Port host	Keterangan
client-facing-api	8080	Pintu masuk client (dapat diubah, mis. CLIENT_API_PORT=8085)
bmkg-mock	8081	Mock BMKG
pvmbg-mock	8082	Mock PVMBG (termasuk /admin/*)
aggregator	8090 (hanya 127.0.0.1)	API baca internal (/hazard-events, /pvmbg-status, /health)
auth-service	8091	POST /token, /public-key, /revocations
dashboard-updater	tidak dipublikasikan (8092 di jaringan Compose)	/health dan /state (diakses lewat docker compose exec)
notifier	tidak dipublikasikan (8093 di jaringan Compose)	/health
RabbitMQ	5672, UI 15672 (hanya localhost)	AMQP dan *management UI*
MongoDB	tidak dipublikasikan	Hanya terjangkau dari aggregator melalui store-net

Tabel 25. Daftar port layanan

10.3 Memicu Scenario

Nilai default dari .env.example. Login Media dan Lapangan memakai media-demo/media-demo-secret dan field-demo/field-demo-secret. Hindari lebih dari beberapa *login* gagal per menit karena *throttle* 10 kali per menit per IP akan menahan login berikutnya. Koleksi Postman tersedia di docs/postman-collection.json.

Login dan baca data

```
curl -s -X POST localhost:8091/token -d grant_type=client_credentials \
  -d client_id=ops-demo -d client_secret=ops-demo-secret
T=<access_token>
curl -s -H "Authorization: Bearer $T" 'localhost:8080/hazard-events?limit=5'
```

Problem 1: ubah skema PVMBG tanpa *restart*

```
curl -X POST localhost:8082/admin/schema-version -H "Authorization: Bearer
pvmbg-demo-token" -d '{"enabled":true}'
curl -s -H "Authorization: Bearer $T"
'localhost:8080/hazard-events?hazard_type=VOLCANIC&limit=5'
```

Problem 2: outage dan delay PVMBG

```
curl -X POST localhost:8082/admin/outage -H "Authorization: Bearer
pvmbg-demo-token" -d '{"enabled":true}'
curl -s -H "Authorization: Bearer $T"
'localhost:8080/hazard-events?hazard_type=SEISMIC&limit=3'   # normal
curl -s -H "Authorization: Bearer $T"
'localhost:8080/hazard-events?hazard_type=VOLCANIC&limit=3'  # data lama +
stale_since
curl -X POST localhost:8082/admin/outage -H "Authorization: Bearer
pvmbg-demo-token" -d '{"enabled":false}'

# delay PVMBG 3 detik
PVMBG_DELAY_MIN_MS=3000 PVMBG_DELAY_MAX_MS=3000 docker compose up -d
pvmbg-mock

# load test (k6, naikkan TTL token dulu: AUTH_ACCESS_TTL_SECONDS=300 docker
compose up -d auth-service)
API_TOKEN=$T k6 run tests/k6/p2-sustained.js
API_TOKEN=$T k6 run tests/k6/p2-isolation.js   # setelah delay PVMBG 3 dtk
API_TOKEN=$T PVMBG_TOKEN=pvmbg-demo-token k6 run tests/k6/p2-outage.js

# alternatif cepat
hey -z 60s -c 50 -H "Authorization: Bearer $T"
'http://localhost:8080/hazard-events?hazard_type=SEISMIC'
```

Problem 3: autentikasi dan trust

```
curl -i -H "Authorization: Bearer $MEDIA_T"
'localhost:8080/hazard-events?fields=attributes'   # 403
```

```
curl -s -X POST localhost:8091/token -d grant_type=refresh_token -d
refresh_token=<refresh_token>
curl -i -H "X-BMKG-Key: bmkg-demo-key" localhost:8082/volcanic-reports
      # 401
curl -i -H "Authorization: Bearer pvmbg-demo-token"
localhost:8081/seismic-events              # 401
curl -i -H "X-BMKG-Key: pvmbg-demo-token" localhost:8081/seismic-events
      # 403
```

Problem 4: microservice dan storage

```
docker compose up -d --build --no-deps auth-service
docker compose exec notifier nc -zv mongo 27017         # gagal
docker compose exec mongo mongosh -u "$MONGO_ROOT_USERNAME" -p
"MONGO_ROOT_PASSWORD" --authenticationDatabase admin \
  --eval
'db.getSiblingDB("bnpb_canonical").hazard_events.findOne({"attributes.confiden
ce_level":{$exists:true}})'
```

Problem 5: pub-sub

```
docker compose stop notifier
sleep 30
docker compose start notifier
docker compose logs notifier | grep -E "backlog_on_start|redelivered"
docker compose exec dashboard-updater wget -qO- localhost:8092/state
```

Bukti correlation ID lintas service

```
docker compose logs aggregator | grep "hazard event ingested" | tail -1   #
ambil correlation_id
docker compose logs | grep <correlation_id>
```

LAMPIRAN

A. Deklarasi Penggunaan LLM

Dalam pengerjaan tugas besar ini, kelompok menggunakan LLM, sebagai alat bantu untuk mendiskusikan alternatif rancangan, penyusunan laporan dan dokumentasi, membantu proses debugging, serta memberikan saran awal terhadap implementasi (*scaffolding*) dan skenario pengujian. LLM tidak digunakan sebagai pengganti proses perancangan, pengambilan keputusan, maupun pengujian oleh anggota kelompok.

Setiap kode, konfigurasi, dan teks yang dihasilkan atau disarankan oleh LLM telah ditinjau, disesuaikan dengan kebutuhan sistem, dan diverifikasi kembali oleh kelompok melalui pembacaan kode serta pengujian secara langsung. Seluruh anggota kelompok memahami fungsi, alur kerja, asumsi, dan konsekuensi dari kode yang dimasukkan ke dalam *repository*. Dengan demikian, tanggung jawab atas kebenaran implementasi, hasil pengujian, dan isi laporan tetap sepenuhnya berada pada kelompok.

B. Tautan Repositori dan Tag

Repository: https://github.com/KennethhPoenadi/IF4031_chisli1
Tag: https://github.com/KennethhPoenadi/IF4031_chisli1/releases/tag/milestone-1

C. All Output

```
Waktu mulai : 2026-10-09 14:42:54 WIB
Port client-facing-api pada pengujian ini: 8085 (CLIENT_API_PORT)
SERVICE             STATUS
aggregator          Up 4 minutes (healthy)
auth-service        Up 4 minutes (healthy)
bmkg-mock           Up 4 minutes (healthy)
client-facing-api   Up 4 minutes (healthy)
dashboard-updater   Up 4 minutes (healthy)
mongo               Up 4 minutes (healthy)
notifier            Up 4 minutes (healthy)
pvmbg-mock          Up 4 minutes (healthy)
rabbitmq            Up 4 minutes (healthy)


########## P1. Perubahan skema PVMBG tanpa restart ##########
--- sebelum: 3 event vulkanik terbaru (field attributes)
{"hazard_id":"hz-pvmbg-000048","severity":"AWAS","attributes":{"ash_column_height_m":
747.7,"eruption_count_24h":10}}
{"hazard_id":"hz-pvmbg-000047","severity":"SIAGA","attributes":{"ash_column_height_m"
:3328.2,"eruption_count_24h":1}}
{"hazard_id":"hz-pvmbg-000046","severity":"WASPADA","attributes":{"ash_column_height_
m":2004.6,"eruption_count_24h":11}}

$ pv_post /admin/schema-version '{"enabled":true}'
{"enabled":true}

menunggu 14 detik (satu sampai dua siklus polling)...
--- sesudah: 3 event vulkanik terbaru
{"hazard_id":"hz-pvmbg-000051","severity":"SIAGA","attributes":{"ash_column_height_m"
:1079.7,"confidence_level":0.7,"eruption_count_24h":1}}
```

```
{"hazard_id":"hz-pvmbg-000050","severity":"WASPADA","attributes":{"ash_column_height_
m":3807.7,"confidence_level":0.71,"eruption_count_24h":9}}
{"hazard_id":"hz-pvmbg-000049","severity":"NORMAL","attributes":{"ash_column_height_m
":129.6,"confidence_level":0.8,"eruption_count_24h":9}}
--- Mongo: dokumen vulkanik dengan dan tanpa confidence_level
confidence_level exists=true : 6
confidence_level exists=false : 420

$ pv_post /admin/schema-version '{"enabled":false}'
{"enabled":false}


########## P2. Outage PVMBG, penanda stale, dan pemulihan ##########
--- sebelum outage
{"hazard_id":"hz-pvmbg-000051","pvmbg":{"source_status":"HEALTHY","last_success_at":"
2026-10-09T07:43:13.13062597Z","stale":false}}
{"seismic_terbaru":"hz-bmkg-000049"}

$ pv_post /admin/outage '{"enabled":true}'
{"enabled":true}

menunggu 9 detik...
--- saat outage (baru)
{"hazard_id":"hz-pvmbg-000052","pvmbg":{"source_status":"DEGRADED","last_success_at":
"2026-10-09T07:43:17.241683222Z","stale_since":"2026-10-09T07:43:20.653872709Z","stal
e":false}}
{"seismic_terbaru":"hz-bmkg-000050"}
--- status HTTP permintaan vulkanik saat outage
HTTP 200, 0.013089s

menunggu 35 detik agar melewati ambang stale 30 detik...
--- saat outage (sudah melewati ambang)
{"pvmbg":{"source_status":"DEGRADED","last_success_at":"2026-10-09T07:43:17.241683222
Z","stale_since":"2026-10-09T07:43:20.653872709Z","stale":true}}
{"seismic_terbaru":"hz-bmkg-000054"}

$ pv_post /admin/outage '{"enabled":false}'
{"enabled":false}

menunggu 14 detik untuk pemulihan...
--- sesudah outage dimatikan
{"hazard_id":"hz-pvmbg-000057","pvmbg":{"source_status":"HEALTHY","last_success_at":"
2026-10-09T07:44:12.013434761Z","stale":false}}


########## P3. Kredensial, hak akses, kedaluwarsa, refresh ##########
--- kredensial silang antar instansi (baris status dan pesan)
BMKG dengan nilai token PVMBG di X-BMKG-Key:
HTTP/1.1 403 Forbidden
{"error":"invalid BMKG credential"}

PVMBG dengan kunci BMKG sebagai Bearer:
HTTP/1.1 403 Forbidden
{"error":"invalid PVMBG credential"}

BMKG dengan token PVMBG di header Authorization (skema salah):
HTTP/1.1 401 Unauthorized
{"error":"missing X-BMKG-Key"}

PVMBG dengan kunci BMKG di header X-BMKG-Key (skema salah):
HTTP/1.1 401 Unauthorized
{"error":"missing bearer token"}

--- client Media: field yang diterima dan permintaan field mentah
{"area_name":"Gunung
Merapi","hazard_id":"hz-pvmbg-000057","hazard_type":"VOLCANIC","ingested_at":"2026-10
-09T07:44:12.013Z","occurred_at":"2026-10-09T07:44:06.806Z","severity":"NORMAL","sour
ce":"PVMBG"}

$ curl -s -i -H "Authorization: Bearer <token>"
```

```
"http://localhost:8085/hazard-events?fields=latitude" | sed -n '1p;$p'
HTTP/1.1 403 Forbidden
{"error":"forbidden_field","message":"fields not allowed for this client: latitude"}

$ curl -s -i -H "Authorization: Bearer <token>"
"http://localhost:8085/hazard-events?fields=attributes" | sed -n '1p;$p'
HTTP/1.1 403 Forbidden
{"error":"forbidden_field","message":"fields not allowed for this client:
attributes"}

--- client Tim Lapangan: login, refresh, token lama ditolak, reuse
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "read:summary read:raw",
  "access_token": "(disembunyikan)",
  "refresh_token": "(disembunyikan)"
}
--- refresh:
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "read:summary read:raw",
  "access_token": "(disembunyikan)",
  "refresh_token": "(disembunyikan)"
}
--- token LAMA setelah refresh (masih dalam TTL):
HTTP/1.1 401 Unauthorized
Www-Authenticate: Bearer error="token_revoked"
{"error":"token_revoked","message":"access token was revoked (replaced by a
refresh)"}
--- token BARU:
HTTP/1.1 200 OK
--- refresh token yang sama dipakai lagi (reuse):
HTTP/1.1 401 Unauthorized
{"error":"invalid_grant"}
--- token BARU setelah reuse terdeteksi (sesi dicabut):
HTTP/1.1 401 Unauthorized
Www-Authenticate: Bearer error="token_revoked"
{"error":"token_revoked","message":"access token was revoked (replaced by a
refresh)"}

--- kedaluwarsa alami (TTL default 60 detik), menunggu 63 detik...
sebelum TTL habis:
HTTP 200
sesudah TTL habis:
HTTP/1.1 401 Unauthorized
Www-Authenticate: Bearer error="token_expired"
{"error":"token_expired","message":"access token expired, use the refresh token"}


########## P4. Rebuild satu service tanpa menyentuh yang lain ##########
--- waktu mulai tiap container sebelum rebuild
/aat-aggregator-1 2026-10-09T07:38:40.021294677Z
/aat-auth-service-1 2026-10-09T07:38:14.708760804Z
/aat-bmkg-mock-1 2026-10-09T07:38:14.692031137Z
/aat-client-facing-api-1 2026-10-09T07:38:45.992173721Z
/aat-dashboard-updater-1 2026-10-09T07:38:45.98975793Z
/aat-mongo-1 2026-10-09T07:38:14.762463596Z
/aat-notifier-1 2026-10-09T07:38:45.985340555Z
/aat-pvmbg-mock-1 2026-10-09T07:38:14.620776846Z
/aat-rabbitmq-1 2026-10-09T07:38:14.728189221Z

$ docker compose up -d --build --force-recreate --no-deps auth-service 2>&1 | tail -6
#18 DONE 0.0s
 aat-auth-service  Built
 Container aat-auth-service-1  Recreate
 Container aat-auth-service-1  Recreated
 Container aat-auth-service-1  Starting
 Container aat-auth-service-1  Started
```

```
--- kode HTTP loop selama rebuild (ringkasan)
HTTP 200: 70 kali
--- waktu mulai tiap container sesudah rebuild (hanya auth-service yang berubah)
/aat-aggregator-1 2026-10-09T07:38:40.021294677Z
/aat-auth-service-1 2026-10-09T07:45:27.058989004Z
/aat-bmkg-mock-1 2026-10-09T07:38:14.692031137Z
/aat-client-facing-api-1 2026-10-09T07:38:45.992173721Z
/aat-dashboard-updater-1 2026-10-09T07:38:45.98975793Z
/aat-mongo-1 2026-10-09T07:38:14.762463596Z
/aat-notifier-1 2026-10-09T07:38:45.985340555Z
/aat-pvmbg-mock-1 2026-10-09T07:38:14.620776846Z
/aat-rabbitmq-1 2026-10-09T07:38:14.728189221Z
--- notifier mencoba menjangkau mongo
nc: bad address 'mongo'
(kode keluar 1)
--- aggregator memeriksa dirinya (health, termasuk mongo)
{"mongo":true,"rabbitmq":true,"status":"ok"}



########## P5. Consumer dimatikan lalu dinyalakan lagi ##########

$ docker compose exec -T rabbitmq rabbitmqctl list_queues name messages consumers
2>/dev/null | tail -n +3
name    messages       consumers
dashboard-updater-queue      0      1
notifier-queue 0      1

$ docker compose stop notifier 2>&1 | tail -2
 Container aat-notifier-1  Stopping
 Container aat-notifier-1  Stopped
menunggu 35 detik sementara aggregator tetap menerbitkan event...

$ docker compose exec -T rabbitmq rabbitmqctl list_queues name messages consumers
2>/dev/null | tail -n +3
name    messages       consumers
dashboard-updater-queue      0      1
notifier-queue 11     0
--- dashboard-updater tetap menerima event saat notifier mati
jumlah event diterima dashboard-updater dalam 40 detik: 9

$ docker compose start notifier 2>&1 | tail -1
 Container aat-notifier-1  Started
--- log notifier setelah hidup (backlog_on_start)
{"time":"2026-10-09T07:46:41.31598351Z","level":"INFO","msg":"notifier
listening","service":"notifier","exchange":"hazard_events","queue":"notifier-queue","
binding_keys":["hazard.#"],"min_alert_severity":"WASPADA","backlog_on_start":12}

$ docker compose exec -T rabbitmq rabbitmqctl list_queues name messages consumers
2>/dev/null | tail -n +3
name    messages       consumers
dashboard-updater-queue      0      1
notifier-queue 0      1


########## Log latensi panggilan keluar (satu contoh per jenis) ##########
{"time":"2026-10-09T07:38:40.769537094Z","level":"INFO","msg":"bmkg
call","service":"aggregator","path":"/seismic-events","status":200,"duration_ms":136,
"correlation_id":"e234222f3924c101a1b83d33bec57d0f"}
{"time":"2026-10-09T07:38:43.48298597Z","level":"INFO","msg":"pvmbg
call","service":"aggregator","status":200,"duration_ms":2854,"correlation_id":"254bce
50ea9fde57f454dc9379bda576"}
{"time":"2026-10-09T07:38:40.896723636Z","level":"INFO","msg":"mongodb
call","service":"aggregator","target":"mongodb","operation":"save_hazard_event","dura
tion_ms":16,"success":true,"hazard_id":"hz-bmkg-000001","correlation_id":"e234222f392
4c101a1b83d33bec57d0f"}
{"time":"2026-10-09T07:40:30.782157756Z","level":"INFO","msg":"mongodb
call","service":"aggregator","target":"mongodb","operation":"find_hazard_events","dur
ation_ms":37,"success":true,"correlation_id":"9ff86007d410b12b"}
{"time":"2026-10-09T07:38:45.682855638Z","level":"INFO","msg":"mongodb
```

```
call","service":"aggregator","target":"mongodb","operation":"ping","duration_ms":4,"s
uccess":true,"correlation_id":"027188df630fdcc0db00b0b18e8e1f5b"}
{"time":"2026-10-09T07:38:40.899001552Z","level":"INFO","msg":"rabbitmq
call","service":"aggregator","target":"rabbitmq","operation":"publish","duration_ms":
2,"success":true,"hazard_id":"hz-bmkg-000001","correlation_id":"e234222f3924c101a1b83
d33bec57d0f"}
{"time":"2026-10-09T07:40:30.783857214Z","level":"INFO","msg":"upstream
call","service":"client-facing-api","target":"aggregator","status":200,"duration_ms":
65,"correlation_id":"9ff86007d410b12b"}
{"time":"2026-10-09T07:38:46.417868097Z","level":"INFO","msg":"auth-service
call","service":"client-facing-api","target":"auth-service","operation":"get_public_k
ey","duration_ms":11,"correlation_id":"e96678bfb92b9d4a","status":200,"success":true}
{"time":"2026-10-09T07:38:46.680886263Z","level":"INFO","msg":"auth-service
call","service":"client-facing-api","target":"auth-service","operation":"get_revocati
ons","duration_ms":6,"correlation_id":"b984e6403b04f4db","status":200,"success":true}

Waktu selesai: 2026-10-09 14:47:02 WIB
```